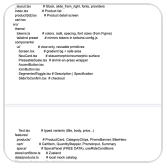
You said:




For the UI component I have to make the designs attached. I have organized my files and folders in a way I think is modular and maintainable. Would you suggest any tweaks in my setup? If not what would be the first logical part to start with (React Native). We are using Tailwind CSS. I currently have the Screen.tsx below

```
import { PropsWithChildren } from 'react';
import { View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';

type ScreenProps = PropsWithChildren<{
  className?: string;
}>;

export function Screen({ children, className = '' }: ScreenProps) {
  return (
    <View className="flex-1 bg-bg">
      <SafeAreaView className={`flex-1 ${className}`}>
{children}</SafeAreaView>
    </View>
  );
}
```

ChatGPT said:

Your folder structure is already well organized. For a React Native application using Expo Router, TypeScript, and Tailwind (NativeWind), I think you've made the right architectural choices.

In particular, I like that you've separated:

- `components/ui/` — Reusable, presentation-only components.
- `features/` — Components specific to products, cart, and specifications.
- `theme/` — Centralized design tokens.
- `store/` — Shared application state using Zustand.
- `data/` — Mock product data independent of the UI.

- `app/` — Routing and screen composition rather than business logic.

For a UI implementation task, this demonstrates that you're thinking beyond just reproducing Figma screens.

I would make a few small adjustments before starting.

1. Suggested folder structure

I'd keep approximately 90% of what you have.

SUGGESTED STRUCTURE

```
app/
  _layout.tsx
  index.tsx
  product/[id].tsx
  cart.tsx

src/
  theme/
    tokens.ts

  components/
    ui/
      Screen.tsx
      NeuCard.tsx
      PressableScale.tsx
      AccentButton.tsx
      IconButton.tsx
      SegmentedToggle.tsx
      SlideToConfirm.tsx
      Text.tsx

  features/
    products/
      components/
        ProductCard.tsx
        CategoryChips.tsx
        PromoBanner.tsx
        BikeHero.tsx
      data/
```

```
        products.ts
      types.ts

    cart/
      components/
        CartItem.tsx
        QuantityStepper.tsx
        PromoInput.tsx
        CartSummary.tsx
      store/
        cartStore.ts

    specs/
      components/
        SpecsPanel.tsx
      data/
        specifications.ts

  lib/
    responsive.ts

tailwind.config.js
```

The main changes are:

A. Put product data close to the product feature.

Rather than `src/data/products.ts`, I'd use `src/features/products/data/products.ts`.

This is a feature-first organization. Everything needed to understand the product feature lives together.

B. Move Zustand into the cart feature.

`src/features/cart/store/cartStore.ts` makes sense because the store specifically manages cart functionality. If you later add authentication or preferences, those can have their own stores.

C. Don't create a separate Tailwind preset unless necessary.

I would start with `tokens.ts` and `tailwind.config.js`, keeping the tokens synchronized. A separate preset is useful when sharing a theme across multiple apps, but may be unnecessary for this project.

D. Keep feature components out of `app/`.

Your existing structure already follows this principle. Expo Router files should mostly compose screens, connect state, and handle navigation.

One additional point: I wouldn't create every file upfront. Add components as you encounter genuine reuse. For example, `CartSummary` and `Summary` don't necessarily need to be separate abstractions.

2. Your Screen.tsx is good, but I'd improve it slightly

Your current implementation:

```
import { PropsWithChildren } from 'react';
import { View } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';

type ScreenProps = PropsWithChildren<{
  className?: string;
}>;

export function Screen({ children, className = '' }: ScreenProps) {
  return (
    <View className="flex-1 bg-bg">
      <SafeAreaView className={`flex-1 ${className}`}>{children}
</SafeAreaView>
    </View>
  );
}
```

This is a perfectly valid starting point.

There are two things worth considering based on your Figma designs.

A. Safe-area handling

Your designs use full-screen backgrounds, with content constrained by safe areas. You also have a bottom navigation bar and fixed bottom purchasing controls.

I'd add a configurable `edges` prop so each screen can control which safe areas it respects.

```
import { PropsWithChildren } from 'react';
import { View } from 'react-native';
import {
  SafeAreaView,
  type Edge,
} from 'react-native-safe-area-context';

type ScreenProps = PropsWithChildren<{
  className?: string;
  edges?: Edge[];
}>;

export function Screen({
  children,
  className = '',
  edges = ['top', 'bottom'],
}: ScreenProps) {
  return (
    <View className="flex-1 bg-bg">
      <SafeAreaView
        edges={edges}
        className={`flex-1 ${className}`}
      >
        {children}
      </SafeAreaView>
    </View>
  );
}
```

This gives you flexibility:

```
<Screen edges={['top']}>
  {/* Product detail */}
</Screen>
```

You may want the product's bottom purchasing controls to handle their own safe-area inset rather than applying padding to the entire screen.

B. Gradients

Your Figma designs rely heavily on blue gradients, particularly behind the product photography.

I would **not** build that into `Screen.tsx`.

Instead, let `Screen` handle screen layout and background color. Add gradients to the specific components that need them, using `expo-linear-gradient` or another suitable gradient implementation.

That makes `Screen` reusable without tying it to this particular visual style.

3. What should you build first?

I would start with **the design tokens, followed immediately by `NeuCard.tsx`.**

Not the product list. Not the navigation. Not the cart.

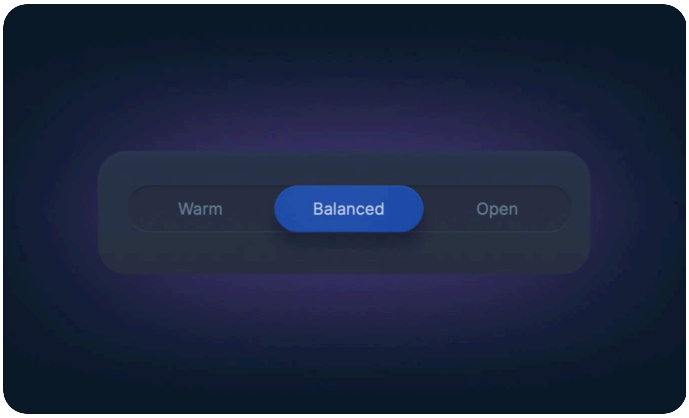
The hardest visual aspect of your Figma design isn't the screen layout. It's the combination of:

- Dark navy surfaces
- Soft, subtle highlights and shadows
- Rounded corners
- Bright blue gradients
- Raised and recessed controls
- Press animations

These details establish the visual language for virtually every screen.



Raised surfaces: product cards, panels, buttons



Recessed surfaces: toggles, quantity controls

Step 1 — Define your design tokens

Start with a small `tokens.ts` .

These are illustrative values, not exact measurements extracted from Figma. Use Figma's inspect panel for the final values.

```
export const colors = {
  bg: '#242C3C',
  surface: '#293246',
  surfaceLight: '#354159',

  accent: '#36B9F4',
  accentDark: '#3458EA',

  text: '#FFFFFF',
  textMuted: '#A8B0C0',

  shadowDark: '#161D2B',
  shadowLight: '#3B4659',
} as const;

export const radii = {
  sm: 8,
  md: 12,
  lg: 20,
  xl: 28,
} as const;

export const spacing = {
```

```
  xs: 4,
  sm: 8,
  md: 16,
  lg: 24,
  xl: 32,
} as const;
```

Then map the appropriate colors into your NativeWind configuration.

For example, `bg-bg` and `bg-surface` should refer to these design values.

A detail worth remembering: NativeWind class names work for many styles, but gradients and more advanced shadow effects will often require dedicated React Native components or styles.

Step 2 — Build `NeuCard.tsx`

This should be your first genuinely reusable visual component.

I'd give it a straightforward API:

```
<NeuCard variant="raised">
  <Text>Product content</Text>
</NeuCard>
```

And potentially:

```
<NeuCard variant="inset">
  <Text>Recessed content</Text>
</NeuCard>
```

Keep the initial props small:

```
type NeuCardProps = PropsWithChildren<{
  variant?: 'raised' | 'inset';
  className?: string;
}>;
```

One implementation detail: React Native shadows behave differently on iOS and Android, and CSS-style dual shadows do not translate directly across both platforms.

For a strong neumorphic effect, you may need nested layers or platform-specific shadow styling. I would test it on an actual device or simulator before building every other component around it.

Step 3 — Build `AccentButton.tsx`

This is the most obvious shared component across your designs.



You already identified the press animation requirement. I'd implement the scale animation in `PressableScale.tsx` and reuse that inside your button.

I'd use React Native Reanimated if it's already in your Expo project, with a short spring animation that scales the button to roughly `0.95` on press and returns to `1` on release.

This creates a reusable interaction primitive rather than repeating animation logic everywhere.

4. Recommended implementation order

Here's the sequence I'd follow, prioritizing dependencies and getting visible progress early.

Build checklist 0 of 6 complete

- [ ] **1. Foundation**
  Design tokens, typography, Screen, font loading
- [ ] **2. UI primitives**
  NeuCard, PressableScale, AccentButton, IconButton
- [ ] **3. Product listing**
  ProductCard, CategoryChips, PromoBanner, mock catalog
- [ ] **4. Product details**
  BikeHero, SegmentedToggle, SpecsPanel, bottom purchase bar

- [ ] **5. Cart**
  Zustand store, CartItem, QuantityStepper, PromoInput, totals
- [ ] **6. Motion and polish**
  Navigation transitions, slide-to-confirm, shadow and spacing refinements

One note on navigation: implement the basic Expo Router stack early enough to test your screens, but save polished transitions for the final stage. Expo Router supports push navigation, and you can configure a right-to-left screen entrance using stack animation options. The checkout interaction within the cart should be animated separately because it has a different motion requirement.

5. One thing I'd avoid: overengineering

Because this is a take-home UI task, I would be careful not to spend too much time building a design system before producing actual screens.

A useful rule is:

Make a component reusable when it appears more than once or has meaningful behavior worth isolating.

For example:

Component	Worth extracting?	Reason
`Screen`	Yes	Used on every screen
`NeuCard`	Yes	Shared visual treatment
`AccentButton`	Yes	Consistent colors and press behavior
`PressableScale`	Yes	Reusable interaction behavior
`BikeHero`	Yes	Encapsulates the product image presentation
`SpecsPanel`	Yes	Contains a distinct UI section
`PromoBanner`	Maybe	Extract if it simplifies the listing screen
`Text`	Maybe	Useful for standardized typography, but not strictly necessary

My recommendation

Your architecture is strong enough to begin. I wouldn't make any major structural changes.

I'd do the following next:

1. Finalize `theme/tokens.ts` using the Figma values.
2. Keep your updated `Screen.tsx`.
3. Implement and visually verify `NeuCard.tsx`.
4. Build `AccentButton.tsx` and its press animation.
5. Then start the product listing screen.

That sequence gives you reusable building blocks without losing momentum.

One question before you implement `NeuCard`: **Are you using NativeWind v4 or v5?** The answer matters for the exact Tailwind configuration and how I would implement your neumorphic shadows.

You said:

‹/› **AccentButton.tsx** Code	‹/› **PressableScale.tsx** Code	‹/› **NeuCard.tsx** Code

Would you say these are solid for now? I think we should move on to the product listing screen, using the same code style I did here

ChatGPT said:

Yes. **These three components are solid foundations, and I think you're ready to move on to the product listing screen.**

I reviewed `AccentButton.tsx`, `PressableScale.tsx`, and `NeuCard.tsx`. Your code is readable, concise, and appropriately modular for a React Native take-home project.

There are only a few minor things I'd flag:

Component	Assessment	Minor improvement
`AccentButton`	Good	The fixed 130 × 36 dimensions may eventually need to be configurable.
`PressableScale`	Good	Replace `style as any` with proper typing later.

`NeuCard`	Good	Verify that `boxShadow` renders correctly on your target device.

One thing worth fixing now: `AccentButton` passes `className` into `PressableScale`, but `PressableScale` doesn't explicitly include `className` in its props. Depending on your NativeWind setup, TypeScript may reject it. You can add `className?: string` to its `Props` type.

Otherwise, I wouldn't spend more time refining these components before you have a working screen.

Let's build the product listing screen

Based on your Figma, the first screen has five main sections:

1. Header — "Choose Your Bike" and search button.
2. Promotional card — featured bicycle with a 30% discount.
3. Category selector — horizontally arranged category icons.
4. Product grid — cards with images, names, prices, and favorites.
5. Bottom navigation — fixed navigation bar.

I'd build this incrementally rather than writing the entire screen at once.

We can preserve your current coding style: small functional components, simple `Props` types, NativeWind classes for layout, and minimal inline styles for visual effects that NativeWind doesn't handle easily.

Step 1 — Build the screen layout and category selection

First, let's create `app/index.tsx` with the header, categories, and scrollable content area. We'll add the product cards once the overall layout is correct.

```
import { useState } from 'react';
import { ScrollView, Text, View, Pressable } from 'react-native';
import { Screen } from '@/components/ui/Screen';

const categories = [
  'All',
  'Road',
  'Mountain',
  'Helmet',
```

```
];

export default function ProductListingScreen() {
  const [selectedCategory, setSelectedCategory] = useState('All');

  return (
    <Screen>
      <View className="flex-1 px-4">
        {/* Header */}
        <View className="flex-row items-center justify-between py-4">
          <Text className="text-white text-lg font-bold">
            Choose Your Bike
          </Text>

          <Pressable
            className="h-10 w-10 items-center justify-center rounded-xl bg-
accent"
            onPress={() => {}}
          >
            <Text className="text-white text-lg">⌕</Text>
          </Pressable>
        </View>

        <ScrollView
          showsVerticalScrollIndicator={false}
          contentContainerStyle={{ paddingBottom: 100 }}
        >
          {/* Promotional banner */}
          <View className="mt-4 h-48 rounded-2xl bg-surface p-4">
            <Text className="mt-auto text-white text-xl font-bold">
              30% Off
            </Text>
          </View>

          {/* Categories */}
          <ScrollView
            horizontal
            showsHorizontalScrollIndicator={false}
            className="mt-5"
            contentContainerStyle={{ gap: 12 }}
          >
            {categories.map((category) => (
```

```
              <Pressable
                key={category}
                onPress={() => setSelectedCategory(category)}
                className={`h-12 min-w-12 items-center justify-center rounded-
xl px-3 ${
                  selectedCategory === category
                    ? 'bg-accent'
                    : 'bg-surface'
                }`}
              >
                <Text className="text-white text-xs font-medium">
                  {category}
                </Text>
              </Pressable>
            ))}
          </ScrollView>

          {/* Product grid will go here */}
          <View className="mt-6" />
        </ScrollView>
      </View>
    </Screen>
  );
}
```

This is the structural first pass. The promotional image, category icons, gradients, and bottom navigation are intentionally not implemented yet. The search control is also only a visual placeholder, not a functional search button.

Why start here?

This immediately gives us a working screen where we can verify the safe areas, spacing, scroll behavior, header dimensions, and category interaction.

Once that looks correct, I'd extract the category selector into `CategoryChips.tsx` and build `ProductCard.tsx` using your existing `NeuCard` and `PressableScale` components.

For the product grid, we'll use a two-column layout with staggered vertical positioning to reproduce the Figma. We can use `FlatList` or separate columns depending on whether the cards need to maintain that staggered arrangement as the catalog grows.

One important thing before the next step: Do you already have the bicycle and helmet images exported from Figma? Those will make a significant difference in getting the listing screen pixel-accurate, so I'd prefer using the real assets instead of approximating them.

You said:



product.ts
Code

products.ts
Code

cartStore.ts
Code

Seems like a solid start, the colors look consistent. I added these models to help organize the data. I think given this we should start with the ProductCard feature as well as CategoryChips to separate the category selection ui logic. Should also logically do product/[id] to read the id and lookup the product. The cartStore should also have relevant helper functions in this file (promo code application for example). How should we tackle this?

ChatGPT said:

I think you're approaching this in exactly the right order. Your data models provide a good foundation, and the next logical step is to connect the data to reusable UI components, then introduce navigation and cart state.

After reviewing your three files, I would tackle this in four stages:

1. `CategoryChips.tsx` — Extract category selection into a reusable controlled component.
2. `ProductCard.tsx` — Build the product UI and connect it to your mock catalog.
3. `product/[id].tsx` — Use Expo Router to resolve the selected product.
4. `cartStore.ts` — Implement cart operations, quantities, promo codes, and derived totals.

I'd keep the same style you've been using: small functional components, simple TypeScript props, NativeWind classes, and straightforward functions rather than unnecessary abstractions.

1. A few observations about your existing files

Your `Product` model is well designed for this task:

```
export type Category = 'road' | 'mountain' | 'helmet';

export type Product = {
  id: string;
  name: string;
  category: Category;
  price: number;
  image: ImageSourcePropType;
  description: string;
  specs: { label: string; value: string }[];
};
```

The `ImageSourcePropType` is appropriate for your locally bundled product assets. Your `getProducts()` and `getProductById()` functions are also exactly the kind of data access helpers I'd expect here.

I'd make one small correction in `products.ts`:

```
export function getProductById(id: string): Product | undefined {
  return products.find((p) => p.id === id);
}
```

Use strict equality (`===`) rather than `==` .

Your cart state currently contains only the type definition, so we can design the implementation cleanly around what the UI actually needs.

One architectural principle I'd follow: **keep product data outside Zustand, and only store product IDs, quantities, and the applied promo code inside the cart store.** This avoids duplicating your product catalog in application state.

2. Build `CategoryChips.tsx`

I would make this a **controlled component**, meaning the parent screen owns the selected category and filtering logic.

This keeps `CategoryChips` focused entirely on presentation and user interaction.

`src/features/products/components/CategoryChips.tsx`

```
import { ScrollView, Text } from 'react-native';
import { Category } from '@/types/product';
import { PressableScale } from '@/components/ui/PressableScale';
import { LinearGradient } from 'expo-linear-gradient';
import { gradients } from '@/theme/tokens';

type CategoryOption = Category | 'all';

type Props = {
  selected: CategoryOption;
  onSelect: (category: CategoryOption) => void;
};

const categories: { label: string; value: CategoryOption }[] = [
  { label: 'All', value: 'all' },
  { label: 'Road', value: 'road' },
  { label: 'Mountain', value: 'mountain' },
  { label: 'Helmet', value: 'helmet' },
];

export function CategoryChips({ selected, onSelect }: Props) {
  return (
    <ScrollView
      horizontal
      showsHorizontalScrollIndicator={false}
      contentContainerStyle={{ gap: 12, paddingVertical: 8 }}
    >
      {categories.map((category) => {
        const isSelected = selected === category.value;

        return (
          <PressableScale
            key={category.value}
            onPress={() => onSelect(category.value)}
            accessibilityRole="button"
            accessibilityState={{ selected: isSelected }}
          >
            {isSelected ? (
              <LinearGradient
                {...gradients.accent}
                className="h-12 min-w-12 items-center justify-center rounded-
```

```
xl px-3"
              >
                <Text className="text-xs font-semibold text-white">
                  {category.label}
                </Text>
              </LinearGradient>
            ) : (
              <Text className="h-12 min-w-12 overflow-hidden rounded-xl bg-
surface px-3 text-center text-xs leading-[48px] text-white">
                {category.label}
              </Text>
            )}
          </PressableScale>
        );
      })}
    </ScrollView>
  );
}
```

This is intentionally simple. We can replace the labels with the category icons from Figma later without changing the selection logic.

The screen remains responsible for deciding which products to show.

3. Build `ProductCard.tsx`

Your Figma has a raised, slightly tilted product-card design. I'd first get the content and interactions correct, then add the staggered layout and visual effects.

`src/features/products/components/ProductCard.tsx`

```
import { Image, Text, View } from 'react-native';
import { useRouter } from 'expo-router';
import { Heart } from 'lucide-react-native';

import { Product } from '@/types/product';
import { NeuCard } from '@/components/ui/NeuCard';
import { PressableScale } from '@/components/ui/PressableScale';

type Props = {
  product: Product;
```

```
};

export function ProductCard({ product }: Props) {
  const router = useRouter();

  return (
    <PressableScale
      onPress={() => router.push({
        pathname: '/product/[id]',
        params: { id: product.id },
      })}
      accessibilityRole="button"
      accessibilityLabel={`View ${product.name}`}
      className="flex-1"
    >
      <NeuCard className="overflow-hidden p-3">
        <View className="items-end">
          <Heart size={18} color="#FFFFFF" />
        </View>

        <Image
          source={product.image}
          resizeMode="contain"
          className="h-32 w-full"
        />

        <View className="mt-3">
          <Text className="text-xs text-gray-400">
            {product.category.toUpperCase()}
          </Text>

          <Text
            numberOfLines={2}
            className="mt-1 text-sm font-semibold text-white"
          >
            {product.name}
          </Text>

          <Text className="mt-2 text-xs text-blue-400">
            ${product.price.toLocaleString('en-US', {
              minimumFractionDigits: 2,
              maximumFractionDigits: 2,
```

```
            })}
          </Text>
        </View>
      </NeuCard>
    </PressableScale>
  );
}
```

This component has one responsibility: display a product and navigate to its detail screen when pressed.

I deliberately haven't added favorite state yet. The heart icon is visual for now; when favorites are implemented, it should become a separately accessible button with its own state and event handling.

Also, `lucide-react-native` is an optional icon library here. If you're already using another icon library, keep that one instead.

4. Connect everything in `app/index.tsx`

Now we can replace the category placeholder and add the product grid.

The listing screen owns the selected category and requests the filtered product list.

```
import { useState } from 'react';
import { ScrollView, Text, View } from 'react-native';

import { Screen } from '@/components/ui/Screen';
import { Category } from '@/types/product';
import { getProducts } from '@/data/products';

import { CategoryChips } from '@/features/products/components/CategoryChips';
import { ProductCard } from '@/features/products/components/ProductCard';

type CategoryOption = Category | 'all';

export default function ProductListingScreen() {
  const [selectedCategory, setSelectedCategory] =
    useState<CategoryOption>('all');

  const filteredProducts = getProducts(
```

```
    selectedCategory === 'all' ? undefined : selectedCategory
  );

  return (
    <Screen>
      <View className="flex-1 px-4">
        <View className="flex-row items-center justify-between py-4">
          <Text className="text-lg font-bold text-white">
            Choose Your Bike
          </Text>
        </View>

        <ScrollView
          showsVerticalScrollIndicator={false}
          contentContainerStyle={{ paddingBottom: 100 }}
        >
          {/* PromoBanner will replace this placeholder */}
          <View className="mt-4 h-48 rounded-2xl bg-surface p-4">
            <Text className="mt-auto text-xl font-bold text-white">
              30% Off
            </Text>
          </View>

          <View className="mt-5">
            <CategoryChips
              selected={selectedCategory}
              onSelect={setSelectedCategory}
            />
          </View>

          <View className="mt-6 flex-row flex-wrap gap-3">
            {filteredProducts.map((product) => (
              <View
                key={product.id}
                style={{ width: '48%' }}
              >
                <ProductCard product={product} />
              </View>
            ))}
          </View>
        </ScrollView>
      </View>
```

```
    </Screen>
  );
}
```

This gets filtering and navigation connected with minimal code. For this small mock catalog, mapping products inside a `ScrollView` is perfectly reasonable.

Once we implement the final staggered grid, we can revisit its layout, scrolling performance, and responsiveness.

5. Implement `product/[id].tsx`

I would implement the route next, before spending too much time styling the cards.

That way, clicking a product immediately takes you to the correct data, and we have a complete listing-to-detail flow.

`app/product/[id].tsx`

```
import { Image, Text, View } from 'react-native';
import { useLocalSearchParams, Stack } from 'expo-router';

import { Screen } from '@/components/ui/Screen';
import { AccentButton } from '@/components/ui/AccentButton';
import { getProductById } from '@/data/products';
import { useCartStore } from '@/store/cartStore';

export default function ProductDetailScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();

  const product = getProductById(id);
  const add = useCartStore((state) => state.add);

  if (!product) {
    return (
      <Screen>
        <Text className="p-4 text-white">
          Product not found.
        </Text>
      </Screen>
    );
```

```
  }

  return (
    <Screen>
      <Stack.Screen options={{ title: product.name }} />

      <View className="flex-1 p-4">
        <Image
          source={product.image}
          resizeMode="contain"
          className="h-64 w-full"
        />

        <Text className="mt-6 text-xl font-bold text-white">
          {product.name}
        </Text>

        <Text className="mt-4 text-sm text-gray-400">
          {product.description}
        </Text>

        <View className="mt-auto flex-row items-center justify-between">
          <Text className="text-lg text-blue-400">
            ${product.price.toFixed(2)}
          </Text>

          <AccentButton
            label="Add to Cart"
            onPress={() => add(product.id)}
          />
        </View>
      </View>
    </Screen>
  );
}
```

This isn't the final Figma implementation. It establishes the data lookup, navigation, and add-to-cart interaction first.

Later, we'll replace the body with `BikeHero`, `SegmentedToggle`, and `SpecsPanel`, and build the fixed purchasing footer to match Figma.

6. Implement `cartStore.ts`

Your initial type is good. I'd extend it to support discounts and quantities.

One distinction worth making: **cart state and pricing calculations are related, but not identical.**

Zustand should manage the cart's changing state. Pure helper functions can calculate totals from that state.

For a take-home project with a mock catalog, both can live in `cartStore.ts` without becoming difficult to maintain.

Here's an initial implementation:

```
import { create } from 'zustand';
import { getProductById } from '@/data/products';

type CartItem = {
  productId: string;
  qty: number;
};

type CartState = {
  items: CartItem[];
  promoCode: string | null;

  add: (id: string) => void;
  increment: (id: string) => void;
  decrement: (id: string) => void;
  remove: (id: string) => void;
  applyPromo: (code: string) => boolean;
  clear: () => void;
};

const promoCodes: Record<string, number> = {
  BIKE30: 0.30,
};

export const useCartStore = create<CartState>((set) => ({
  items: [],
```

```
  promoCode: null,

  add: (id) => {
    if (!getProductById(id)) return;

    set((state) => {
      const existing = state.items.find(
        (item) => item.productId === id
      );

      if (existing) {
        return {
          items: state.items.map((item) =>
            item.productId === id
              ? { ...item, qty: item.qty + 1 }
              : item
          ),
        };
      }

      return {
        items: [...state.items, { productId: id, qty: 1 }],
      };
    });
  },

  increment: (id) => {
    set((state) => ({
      items: state.items.map((item) =>
        item.productId === id
          ? { ...item, qty: item.qty + 1 }
          : item
      ),
    }));
  },

  decrement: (id) => {
    set((state) => ({
      items: state.items
        .map((item) =>
          item.productId === id
            ? { ...item, qty: item.qty - 1 }
```

```
            : item
        )
        .filter((item) => item.qty > 0),
    }));
  },

  remove: (id) => {
    set((state) => ({
      items: state.items.filter(
        (item) => item.productId !== id
      ),
    }));
  },

  applyPromo: (code) => {
    const normalized = code.trim().toUpperCase();

    if (!Object.prototype.hasOwnProperty.call(promoCodes, normalized)) {
      return false;
    }

    set({ promoCode: normalized });
    return true;
  },

  clear: () => set({
    items: [],
    promoCode: null,
  }),
}));

export function getCartTotals(
  items: CartItem[],
  promoCode: string | null
) {
  const subtotal = items.reduce((total, item) => {
    const product = getProductById(item.productId);

    return total + (product?.price ?? 0) * item.qty;
  }, 0);

  const discountRate = promoCode
```

```
    ? promoCodes[promoCode] ?? 0
    : 0;

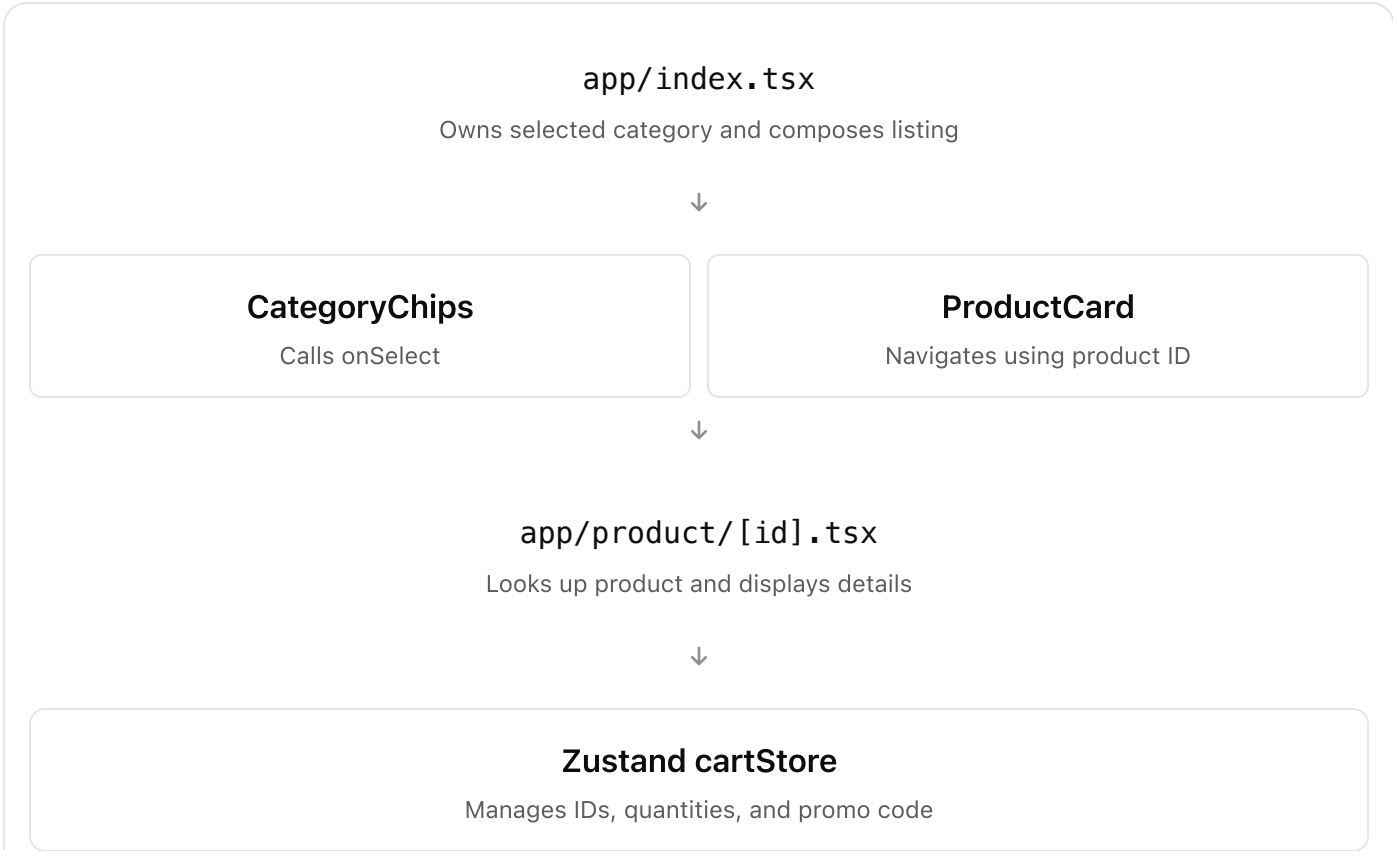
  const discount = subtotal * discountRate;
  const total = subtotal - discount;

  return {
    subtotal,
    discount,
    total,
  };
}
```

This provides everything needed for the basic cart UI. It implements `BIKE30` as a 30% discount on the full subtotal, with no shipping charge. That's a deliberate mock rule, not a verified Figma business requirement. We should verify the expected discount and shipping behavior before completing checkout.

I would also leave persistence out for now unless the task requires carts to survive app restarts.

7. How the pieces fit together



↓

`app/cart.tsx`

Renders cart items and calculated totals

What I would do right now

I'd concentrate on **getting `CategoryChips` and `ProductCard` working inside the listing screen first**, then verify that tapping a product correctly resolves `/product/[id]`.

After that, implement the store and the actual product detail UI.

One small issue I also noticed in your screenshot: your screen background is currently light gray, even though the Figma uses a dark navy background. Your text and banner colors appear correct, so I'd check whether `bg-bg` is resolving correctly in NativeWind before proceeding with the finer styling.

Once these pieces work, we'll have the core of the application connected, and the remaining work will largely be matching Figma accurately rather than designing new application architecture.

You said:

How do I stop unused imports from deleting when I save?

ChatGPT said:

If you're using **VS Code**, unused imports are probably being automatically removed by the **Organize Imports** action when you save.

Option 1: Disable it in VS Code settings (recommended)

1. Open VS Code Settings with `Cmd + ,`.
2. Search for `codeActionsOnSave`.
3. Find **Editor: Code Actions On Save**.
4. Disable **source.organizeImports**.

Option 2: Update settings.json directly

1. Press `Cmd + Shift + P`.
2. Search for **Preferences: Open User Settings (JSON)**.

3. Add or modify:

```
{
  "editor.codeActionsOnSave": {
    "source.organizeImports": "never",
    "source.fixAll": "explicit"
  }
}
```

What this does:

- `source.organizeImports: "never"` — Prevents unused imports from being automatically deleted.
- `source.fixAll: "explicit"` — Runs general fixes only when explicitly requested.

If imports are still disappearing

Check whether ESLint is removing them on save.

Look for:

```
"editor.codeActionsOnSave": {
  "source.fixAll.eslint": "always"
}
```

Change it to:

```
"editor.codeActionsOnSave": {
  "source.fixAll.eslint": "explicit"
}
```

Also check whether an ESLint rule such as `unused-imports/no-unused-imports` is configured to automatically remove unused imports.

My recommendation: Disable `source.organizeImports` but leave TypeScript's unused-import warnings enabled. That way, VS Code still highlights unnecessary imports without deleting ones you're planning to use.

You said:

What is zustand

ChatGPT said:

Zustand is a lightweight state management library for React and React Native. It lets different components in your application access and modify shared data without passing that data through multiple layers of props.

For your bike shopping application, Zustand is a good fit for managing the **shopping cart**.

1. Why do we need Zustand?

Imagine this flow in your application:

1. A user opens a product from the product listing.
2. They tap **Add to Cart**.
3. They navigate to the shopping cart.
4. They increase the quantity from 1 to 2.
5. They apply the `BIKE30` discount code.
6. They navigate back to the product listing.

All these screens need to agree on what's in the cart.

Without a shared state solution, you might need to pass data between screens using props, callbacks, or React Context.

Zustand gives you a central place to manage this information.

2. How Zustand works

A Zustand *store* contains two things:

- **State:** The data your application needs to remember.
- **Actions:** Functions that modify that data.

For example, here's a simplified version of your `cartStore.ts` :

```
import { create } from 'zustand';

type CartState = {
  count: number;
  increment: () => void;
  decrement: () => void;
```

```
};

export const useCartStore = create<CartState>((set) => ({
  count: 0,

  increment: () =>
    set((state) => ({ count: state.count + 1 })),

  decrement: () =>
    set((state) => ({ count: state.count - 1 })),
}));
```

Notice the `create()` function. This creates the store and returns a React hook called `useCartStore`.

The `set()` function updates the state and notifies subscribed components.

3. Using the store in a React Native component

Here's how you would access that store:

```
import { View, Text, Button } from 'react-native';
import { useCartStore } from '@/store/cartStore';

export function CartCounter() {
  const count = useCartStore((state) => state.count);
  const increment = useCartStore((state) => state.increment);
  const decrement = useCartStore((state) => state.decrement);

  return (
    <View>
      <Text>{count} items</Text>

      <Button title="+" onPress={increment} />
      <Button title="-" onPress={decrement} />
    </View>
  );
}
```

Notice that we don't need to pass `count`, `increment`, or `decrement` as props.

Any component can access them directly through the same store.

When `count` changes, React automatically re-renders components subscribed to that value.

4. Zustand versus useState

You already used `useState` in your product listing screen for category selection.

What's the difference?

`useState`	**Zustand**
State belongs to a component	State lives in a shared store
Ideal for local UI interactions	Ideal for data shared across screens
No external library required	Requires the Zustand package
Easy for isolated state	Easy for shared application state

In your application, I would use:

React `useState`

- Currently selected product category
- Description/specification toggle
- Whether a search field is visible
- Temporary text typed into a promo-code field

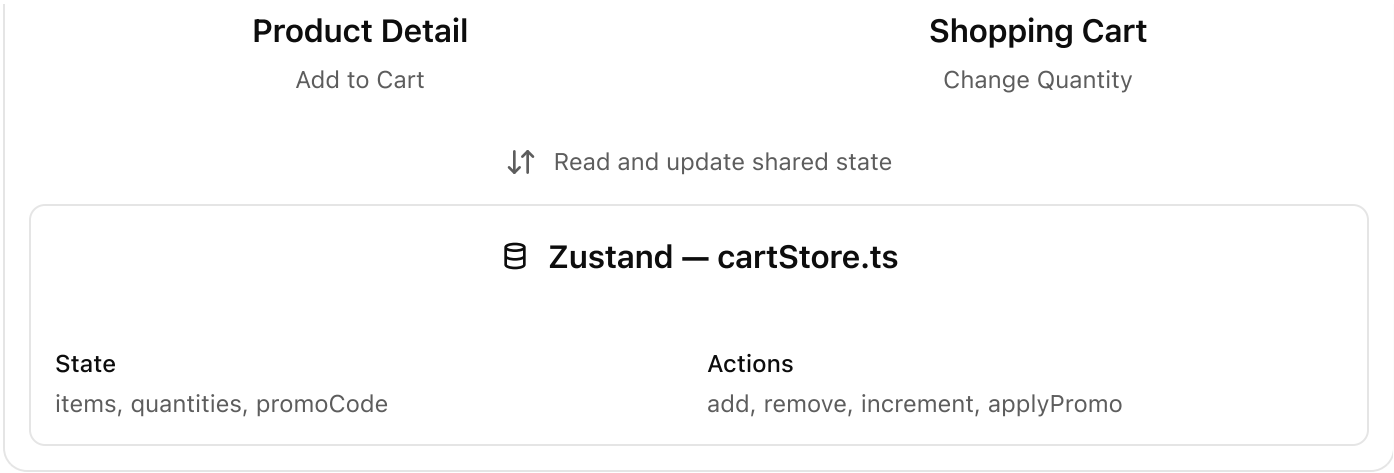
Zustand

- Products currently in the shopping cart
- Quantity of each product
- Applied promo code
- Potentially favorite products

The distinction is that Zustand is most useful when multiple components or screens need access to the same changing data.

5. How your application will use it



For example, when the user clicks Add to Cart:

```
const add = useCartStore((state) => state.add);

<AccentButton
  label="Add to Cart"
  onPress={() => add(product.id)}
/>
```

Zustand updates the shared cart.

When the user navigates to `cart.tsx`, the cart screen reads the updated items:

```
const items = useCartStore((state) => state.items);
```

Both screens access the same data automatically.

One important clarification: **Zustand stores data in memory by default.** If you want the cart to survive closing and reopening the application, you can add Zustand's `persist` middleware with React Native's AsyncStorage.

6. Why I recommended Zustand for your take-home

There are larger state management solutions, including Redux Toolkit, but Zustand has some advantages for a project of this size:

- Very little boilerplate.
- Excellent TypeScript support.

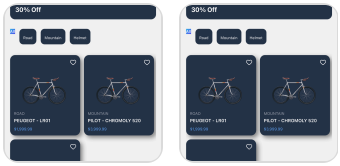
- No provider wrapper required around your application.
- Components can subscribe to specific pieces of state.
- Easy to test because store actions and pricing helpers can be tested independently.

My recommendation: Continue using `useState` for category selection and UI toggles, and Zustand for the shopping cart. This is a practical, maintainable separation that you can easily explain in a technical interview.

An important way to describe it in that interview would be:

> I used local React state for screen-specific interactions and Zustand for shared cart state, allowing the product detail and cart screens to remain synchronized without prop drilling.

You said:



On the right track? I'm just using the same image for all for testing

ChatGPT said:

Yes, **you're definitely on the right track.** The product listing screen is coming together, and your component-based approach is working well.

Comparing your current implementation against the original Figma, you've already established several important pieces:

- **Product cards:** Rounded corners, dark surfaces, images, typography, and prices are all rendering correctly.
- **Two-column grid:** The spacing is consistent, and the cards are aligned.
- **Category selection:** The component is rendering and displaying the selected state.
- **Product data:** Your product names, categories, and prices are coming from the catalog rather than being hardcoded into the UI.
- **Neumorphic shadows:** The raised-card effect is visible.

Using the same bicycle image for all three products is completely fine during development.

What I'd improve next

There are four visual differences I'd address before moving to the product detail screen.

1. Fix the background color first

This is the biggest difference from Figma.

Your current screen has a light gray background, whereas the design uses a dark navy background with blue gradient accents.

The white header text is almost invisible because of this.

Since your `Screen.tsx` already uses `bg-bg`, I'd check that the `bg` token is correctly configured in NativeWind. Also check whether the Expo Router layout or navigation container is supplying the light background.

Getting the background correct will immediately make the screen feel much closer to the design.

2. Reduce the product card dimensions

Your cards are currently quite large relative to the screen.

In the Figma design, they're more compact, allowing more of the catalog to be visible without scrolling.

I'd reduce:

- The product image height.
- The vertical padding.
- The space between the image and product information.

However, don't sacrifice text readability just to match the proportions.

3. Revisit the product grid layout

Your current grid is conventional: two cards side by side, followed by another row.

The Figma uses a more distinctive **staggered arrangement**, with cards positioned at different vertical offsets.

I would eventually recreate that layout because it's an important part of the visual identity.

For a small fixed catalog, you could use two columns with different top padding rather than introducing a complicated masonry layout library.

4. Fix the category buttons

The selected "All" category currently looks like highlighted text rather than a complete button.

I'd check how `LinearGradient` is sizing inside `PressableScale`. Your unselected categories have proper dimensions, but the selected category appears to have lost its container styling.

Remember that the Figma category buttons are small, icon-based controls rather than text labels. We can swap in the appropriate icons after the layout works.

One more important observation

Your shadows are quite strong against the light background. Once you switch to dark navy, they may look more natural, but I'd still compare them closely with Figma.

The design uses relatively subtle neumorphic effects rather than large, dark drop shadows.

What I'd do next

I would **finish the product listing screen before moving to `product/[id]`**.

Specifically, I'd work in this order:

1. Fix the screen background and selected category gradient.
2. Adjust the product card dimensions and spacing.
3. Implement the staggered product layout.
4. Add the featured promotional bicycle image and background gradient.
5. Replace the category labels with icons.

After that, we'll implement the product detail screen, where your `getProductById()` helper and Zustand cart store will come into play.

One thing I particularly like about your progress: You've built the product cards using reusable components and catalog data rather than hardcoding three separate cards. That's exactly the kind of implementation I'd want to see in a React Native take-home submission.

You're past the initial architecture stage now. The next challenge is achieving the visual fidelity of the Figma without compromising the clean structure you've established.

You said:

How would I change the background to match the Figma?

ChatGPT said:

Looking at your Figma, the background is actually more interesting than a solid navy color. It has a **dark navy base with a diagonal blue gradient running through the right side of the screen**.

I'd implement this using `expo-linear-gradient`, which you're already using in `AccentButton.tsx`.

1. First, fix your Screen background

Your current `Screen.tsx` uses:

```
<View className="flex-1 bg-bg">
```

Since your screenshot shows a light gray background, `bg-bg` may not be resolving correctly through NativeWind.

First, verify this by temporarily changing it to:

```
<View className="flex-1 bg-[#242C3C]">
```

If the screen becomes navy, the issue is your Tailwind color configuration.

2. Add a background gradient to Screen.tsx

I would update your `Screen.tsx` like this:

```
import { PropsWithChildren } from 'react';
import { View } from 'react-native';
import {
  SafeAreaView,
  type Edge,
} from 'react-native-safe-area-context';
import { LinearGradient } from 'expo-linear-gradient';

type ScreenProps = PropsWithChildren<{
  className?: string;
  edges?: Edge[];
}>;

export function Screen({
  children,
  className = '',
  edges = ['top', 'bottom'],
}: ScreenProps) {
  return (
    <View className="flex-1 bg-[#242C3C]">
      <LinearGradient
        colors={['#242C3C', '#2D426B', '#3458EA']}
        locations={[0, 0.6, 1]}
        start={{ x: 0, y: 0 }}
        end={{ x: 1, y: 1 }}
        style={{
          position: 'absolute',
          top: 0,
          bottom: 0,
          left: 0,
          right: 0,
```

```
        }}
      />

      <SafeAreaView
        edges={edges}
        className={`flex-1 ${className}`}
      >
        {children}
      </SafeAreaView>
    </View>
  );
}
```

What this does:

- `#242C3C` ▢ establishes your dark navy background.
- `#2D426B` ▢ introduces a subtle blue transition.
- `#3458EA` ▢ creates the brighter blue accent.
- `start` and `end` control the gradient direction.
- Absolute positioning ensures the gradient covers the entire screen, including behind the safe areas.

The colors are approximate. You should replace them with the exact values from Figma once you inspect the background layers.

3. One important adjustment for your Figma

Looking at the original design, the entire screen isn't uniformly blue.

The brightest blue appears around the right side of the promotional banner and product grid, while the left side remains dark navy.

I would actually keep `Screen.tsx` relatively neutral and make the more dramatic gradient specific to the product listing screen.

That avoids forcing the same background treatment onto your product detail and cart screens.

You could add an optional prop:

```
type ScreenProps = PropsWithChildren<{
  className?: string;
  edges?: Edge[];
  gradient?: boolean;
}>;
```

And conditionally render the gradient:

```
{gradient && (
  <LinearGradient
    colors={['#242C3C', '#2D426B', '#3458EA']}
    locations={[0, 0.6, 1]}
    start={{ x: 0, y: 0 }}
    end={{ x: 1, y: 1 }}
    style={{
      position: 'absolute',
      top: 0,
      bottom: 0,
      left: 0,
      right: 0,
    }}
  />
)}
```

Then in `app/index.tsx`:

```
<Screen gradient>
  {/* Product listing content */}
</Screen>
```

4. Don't forget the status bar

Your screenshot currently has a black status bar on a light background.

Since the design uses a dark background, set the status bar icons to white.

In your root `_layout.tsx`:

```
import { StatusBar } from 'expo-status-bar';

export default function RootLayout() {
  return (
    <>
      <Stack screenOptions={{ headerShown: false }} />
      <StatusBar style="light" />
    </>
  );
}
```

This ensures the time, battery, and Wi-Fi indicators are visible against your dark background.

You should also make sure your bottom tab bar uses the same dark navy background instead of its current light gray.

My recommendation

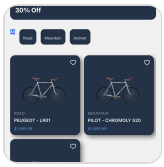
For now, I'd implement the background in two stages:

First: Set `Screen.tsx` to a solid `#242C3C` and verify the entire screen becomes navy.

Second: Add the gradient, but don't make it too bright. The original Figma relies on contrast between dark surfaces and concentrated blue highlights, not a bright blue background across the whole screen.

Once this is correct, your existing product cards should immediately look much closer to the original design.

You said:



Its coming along. I think the next logical step is to match the background, I think a LinearGradient would work here. Would applying a linear gradient in the screen.tsx be logical? How would this look?

ChatGPT said:

Yes. **Applying a `LinearGradient` inside `Screen.tsx` is a good architectural choice**, especially because your product listing, product detail, and cart screens share the same dark visual theme.

However, I'd make the gradient **optional**, so your `Screen` component remains reusable.

The key is to render the gradient *behind* the screen content, rather than wrapping the content inside it.

1. Update `Screen.tsx`

Here's how I'd implement it while keeping your existing code style:

```
import { PropsWithChildren } from 'react';
import { View } from 'react-native';
import {
  SafeAreaView,
```

```
  type Edge,
} from 'react-native-safe-area-context';
import { LinearGradient } from 'expo-linear-gradient';

type Props = PropsWithChildren<{
  className?: string;
  edges?: Edge[];
  gradient?: boolean;
}>;

export function Screen({
  children,
  className = '',
  edges = ['top', 'bottom'],
  gradient = false,
}: Props) {
  return (
    <View className="flex-1 bg-[#242C3C]">
      {gradient && (
        <LinearGradient
          colors={['#242C3C', '#2D426B', '#3458EA']}
          locations={[0, 0.6, 1]}
          start={{ x: 0, y: 0 }}
          end={{ x: 1, y: 1 }}
          style={{
            position: 'absolute',
            top: 0,
            bottom: 0,
            left: 0,
            right: 0,
          }}
        />
      )}

      <SafeAreaView
        edges={edges}
        className={`flex-1 ${className}`}
      >
        {children}
      </SafeAreaView>
    </View>
```

```
  );
}
```

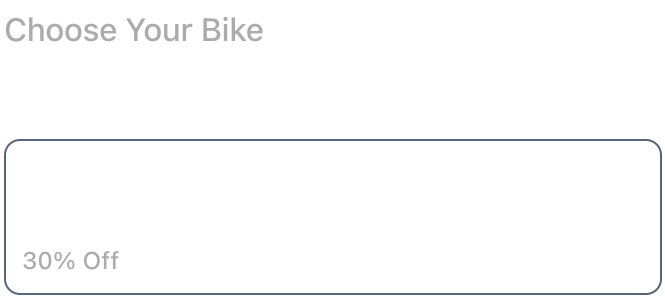
This gives you two options:

```
// Product listing: gradient background
<Screen gradient>
  {/* Content */}
</Screen>

// Other screens: solid navy background
<Screen>
  {/* Content */}
</Screen>
```

2. Matching the Figma more accurately

Looking again at the original design, the background isn't simply a uniform diagonal gradient across the entire screen.



Illustrative color treatment, not exact Figma measurements.

Your Figma has a concentrated blue area toward the right, particularly around the promotional banner and product cards.

A full-screen linear gradient can approximate this, but it won't reproduce the angled transition precisely.

For now, I'd start with a much subtler version:

```
<LinearGradient
  colors={['#242C3C', '#293954', '#3159A3']}
  locations={[0, 0.55, 1]}
  start={{ x: 0, y: 0 }}
  end={{ x: 1, y: 1 }}
  style={{
    position: 'absolute',
    top: 0,
    bottom: 0,
    left: 0,
    right: 0,
  }}
/>
```

This should give you a more controlled transition without overpowering the rest of the UI.

3. Keep the gradient outside your ScrollView

Your component hierarchy should look like this:

```
Screen
  ├── Background LinearGradient
  └── SafeAreaView
        └── ProductListingScreen content
              ├── Header
              └── ScrollView
                    ├── PromoBanner
                    ├── CategoryChips
                    └── ProductCards
```

This means the background remains stationary while your products scroll over it.

It's also important that you aren't applying an opaque background color to the `ScrollView` or its parent, because that would hide the gradient.

4. Your light status bar and bottom navigation

In your current screenshot, the light gray background still covers the status bar and bottom tab area.

Changing `Screen.tsx` should fix the content background, but you'll also want to verify your root `_layout.tsx`.

For example:

```
import { Stack } from 'expo-router';
import { StatusBar } from 'expo-status-bar';

export default function RootLayout() {
  return (
    <>
      <Stack
        screenOptions={{
          headerShown: false,
          contentStyle: {
            backgroundColor: '#242C3C',
          },
        }}
      />
      <StatusBar style="light" backgroundColor="#242C3C" />
    </>
  );
}
```

If you're using Expo Router tabs, configure the tab bar's background separately. The `Stack` settings won't automatically style a tab navigator.

My recommendation: Implement the optional `gradient` prop now, then focus on matching the Figma background before adding more components.

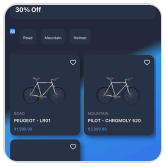
One thing I would change later, though: rather than hardcoding these colors in `Screen.tsx`, add a `screen` gradient to your existing `gradients` object in `theme/tokens.ts`. That keeps the design centralized, just like you've already done with `AccentButton`.

You said:



Test

You said:



Its coming along nicely here. Just need to finish this listing screen to make it look like the FIgma with the featured bicycle image, discount label and the gradient (gradient seems almost there). Also the sizing, shadows and typography in the ProductCard should be adjusted.

After this lets look into the product/[id] (this may be done) as well as the BikeHero.tsx displaying the large product image and the blue diagonal rectangle in the background. Along with this SpecsPanel for the product display specifications and SegmentedToggle for the animated description/specification switch. Here we also have AccentButton.tsx which will be the "Add to Cart" button and cartStore.ts which will connect this button's functionality.

Does this sound straight forward? If there's no missing connections here lets see how this looks with code

ChatGPT said:

Yes. **This is a straightforward implementation plan, and I don't see any major architectural pieces missing.** You've already established the data models, routing structure, UI primitives, and cart state needed to connect everything.

I'd now focus on completing the two screens, rather than introducing any more infrastructure.

There are three small connections worth including:

- `PromoBanner.tsx` should own the featured bicycle presentation, but not the product catalog or discount calculations.
- `BikeHero.tsx` should receive a product as a prop. That keeps `product/[id].tsx` responsible for fetching the correct product.
- `SegmentedToggle.tsx` should receive its selected value and an `onChange` callback, just like `CategoryChips`. This keeps the animation reusable.

For the final product listing, your dark-to-blue background is already much closer to the Figma. The next changes should target the individual elements rather than rebuilding the background.

1. Final component structure

```
app/
  index.tsx
  product/
    [id].tsx
  cart.tsx

src/
  components/
    ui/
      Screen.tsx
      NeuCard.tsx
      PressableScale.tsx
      AccentButton.tsx
      SegmentedToggle.tsx

  features/
    products/
      components/
        ProductCard.tsx
        CategoryChips.tsx
        PromoBanner.tsx
        BikeHero.tsx

    specs/
      components/
        SpecsPanel.tsx

    cart/
```

```
      store/
        cartStore.ts
```

We can do this in two passes: first finish the listing screen, then connect and polish the product detail screen.

2. Finish the product listing

`PromoBanner.tsx`

The banner should contain the featured bicycle image, discount label, and its own subtle background styling.

It can reuse the same `Product` model as your product cards.

```
import { Image, Text, View } from 'react-native';
import { LinearGradient } from 'expo-linear-gradient';
import { useRouter } from 'expo-router';

import { Product } from '@/types/product';
import { PressableScale } from '@/components/ui/PressableScale';

type Props = {
  product: Product;
  discountPercent?: number;
};

export function PromoBanner({
  product,
  discountPercent = 30,
}: Props) {
  const router = useRouter();

  return (
    <PressableScale
      onPress={() =>
        router.push({
          pathname: '/product/[id]',
          params: { id: product.id },
        })
      }
```

```
      scaleTo={0.98}
    >
      <View className="h-48 overflow-hidden rounded-2xl">
        <LinearGradient
          colors={['#303B50', '#273950', '#2966B8']}
          start={{ x: 0, y: 0 }}
          end={{ x: 1, y: 1 }}
          className="flex-1"
        >
          <Image
            source={product.image}
            resizeMode="contain"
            className="absolute inset-0 h-full w-full"
          />

          <View className="flex-1 justify-end p-4">
            <Text className="text-xl font-bold text-white">
              {discountPercent}% Off
            </Text>
          </View>
        </LinearGradient>
      </View>
    </PressableScale>
  );
}
```

The banner's image will be layered over the gradient while the discount stays at the bottom.

You can use the Peugeot bicycle temporarily, although the Figma uses a different featured bike. Eventually, substitute the exported featured bicycle asset.

`ProductCard.tsx`

From your screenshot, I'd prioritize a slightly smaller image and more compact typography, plus less pronounced shadows.

Here's a revised version based on the same components you've already built:

```
import { Image, Text, View } from 'react-native';
import { useRouter } from 'expo-router';
import { Heart } from 'lucide-react-native';
```

```
import { Product } from '@/types/product';
import { NeuCard } from '@/components/ui/NeuCard';
import { PressableScale } from '@/components/ui/PressableScale';

type Props = {
  product: Product;
};

export function ProductCard({ product }: Props) {
  const router = useRouter();

  return (
    <PressableScale
      scaleTo={0.97}
      onPress={() =>
        router.push({
          pathname: '/product/[id]',
          params: { id: product.id },
        })
      }
      className="flex-1"
    >
      <NeuCard className="min-h-52 p-3">
        <View className="items-end">
          <Heart size={17} color="#FFFFFF" strokeWidth={1.8} />
        </View>

        <Image
          source={product.image}
          resizeMode="contain"
          className="h-24 w-full"
        />

        <View className="mt-2">
          <Text className="text-[10px] text-gray-400">
            {product.category.toUpperCase()}
          </Text>

          <Text
            numberOfLines={2}
            className="mt-1 text-xs font-semibold text-white"
```

```
          >
            {product.name}
          </Text>

          <Text className="mt-1 text-xs text-blue-400">
            ${product.price.toLocaleString('en-US', {
              minimumFractionDigits: 2,
              maximumFractionDigits: 2,
            })}
          </Text>
        </View>
      </NeuCard>
    </PressableScale>
  );
}
```

Two important notes: the product image should eventually have its own correct transparent-background asset, and the heart is still decorative. If favorites are part of the task, make it a separate interactive button, with accessible labeling and independent state.

For the shadow, I'd modify your existing `shadows.raised` token rather than overriding it inside each `ProductCard`. That maintains the consistency of your design system.

3. Build `BikeHero.tsx`

This is where we reproduce the most distinctive visual element of the product detail screen: the large bicycle image with the blue diagonal background.



The diagonal background shape behind the product image, without the bicycle overlay.

I would make this its own component rather than adding all the positioning logic directly into the product detail screen.

src/features/products/components/BikeHero.tsx

```
import { Image, View } from 'react-native';
import { LinearGradient } from 'expo-linear-gradient';

import { Product } from '@/types/product';

type Props = {
  product: Product;
};

export function BikeHero({ product }: Props) {
  return (
    <View className="h-64 w-full overflow-hidden">
      {/* Diagonal background */}
      <LinearGradient
        colors={['#40B8F3', '#3655EA']}
        start={{ x: 0, y: 0 }}
        end={{ x: 1, y: 1 }}
        style={{
          position: 'absolute',
          top: -90,
          right: -100,
          width: '65%',
          height: 440,
          transform: [{ skewX: '-25deg' }],
        }}
      />

      {/* Product image */}
      <Image
        source={product.image}
        resizeMode="contain"
        className="h-full w-full"
      />
    </View>
  );
}
```

This uses a skewed rectangular gradient behind the image.

The shape is approximate: once you compare it against the Figma on your iPhone simulator, adjust the skew angle, width, and right offset.

For the product details, I would remove horizontal padding from the hero's parent so the blue background can extend to the screen edge.

4. Build `SegmentedToggle.tsx`

This will handle the Description / Specification switching.

Rather than maintaining separate animation logic inside the product screen, we can make this reusable with React Native Reanimated, which you're already using for `PressableScale`.

The animation will move a highlighted selection background horizontally when the user switches between tabs.

`src/components/ui/SegmentedToggle.tsx`

```
import { useState } from 'react';
import { Pressable, Text, View } from 'react-native';
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withTiming,
} from 'react-native-reanimated';

export type DetailTab = 'description' | 'specification';

type Props = {
  value: DetailTab;
  onChange: (value: DetailTab) => void;
};

export function SegmentedToggle({ value, onChange }: Props) {
  const [width, setWidth] = useState(0);
  const position = useSharedValue(value === 'description' ? 0 : 1);

  position.value = withTiming(
```

```
      value === 'description' ? 0 : 1,
      { duration: 220 }
    );

    const animatedStyle = useAnimatedStyle(() => ({
      transform: [
        { translateX: position.value * (width / 2) },
      ],
    }));

    return (
      <View
        onLayout={(event) =>
          setWidth(event.nativeEvent.layout.width)
        }
        className="h-12 flex-row rounded-xl bg-[#20293A] p-1"
      >
        {width > 0 && (
          <Animated.View
            pointerEvents="none"
            style={[
              {
                position: 'absolute',
                top: 4,
                left: 4,
                width: (width - 8) / 2,
                height: 40,
                backgroundColor: '#303C55',
                borderRadius: 9,
              },
              animatedStyle,
            ]}
          />
        )}

        <Pressable
          onPress={() => onChange('description')}
          className="flex-1 items-center justify-center"
        >
          <Text
            className={`text-xs ${
              value === 'description'
```

```
              ? 'font-semibold text-blue-400'
              : 'text-gray-400'
          }`}
        >
          Description
        </Text>
      </Pressable>

      <Pressable
        onPress={() => onChange('specification')}
        className="flex-1 items-center justify-center"
      >
        <Text
          className={`text-xs ${
            value === 'specification'
              ? 'font-semibold text-blue-400'
              : 'text-gray-400'
          }`}
        >
          Specification
        </Text>
      </Pressable>
    </View>
  );
}
```

This keeps the animation entirely within the toggle, while the parent screen controls which content is displayed.

Notice that the component doesn't need to know anything about bicycles or product specifications. It only needs a selected tab and a callback.

For final polish, we can also animate the content transition rather than immediately swapping the description and specifications.

5. Build `SpecsPanel.tsx`

Your product model already contains an array of specifications:

```
specs: { label: string; value: string }[];
```

That makes this component particularly simple.

src/features/specs/components/SpecsPanel.tsx

```
import { Text, View } from 'react-native';
import { Product } from '@/types/product';

type Props = {
  specs: Product['specs'];
};

export function SpecsPanel({ specs }: Props) {
  return (
    <View className="gap-3">
      {specs.map((spec) => (
        <View
          key={spec.label}
          className="flex-row items-start justify-between border-b border-
white/10 py-2"
        >
          <Text className="flex-1 text-xs text-gray-400">
            {spec.label}
          </Text>

          <Text className="flex-1 text-right text-xs text-white">
            {spec.value}
          </Text>
        </View>
      ))}
    </View>
  );
}
```

This is entirely presentation-only, which is exactly what you want.

For the Figma, the specifications area contains a placeholder marked "FREE DATA." I would replace that with your actual mock specifications unless the task explicitly requires the placeholder.

6. Assemble `product/[id].tsx`

Now we can connect everything.

This screen will:

- Read the product ID from Expo Router.
- Find the product using your existing `getProductById` .
- Render the large product image.
- Switch between descriptions and specifications.
- Display the product price.
- Add the product to the Zustand cart store.

`app/product/[id].tsx`

```
import { useState } from 'react';
import { ScrollView, Text, View } from 'react-native';
import { useLocalSearchParams, Stack } from 'expo-router';

import { Screen } from '@/components/ui/Screen';
import { AccentButton } from '@/components/ui/AccentButton';
import {
  SegmentedToggle,
  DetailTab,
} from '@/components/ui/SegmentedToggle';

import { BikeHero } from '@/features/products/components/BikeHero';
import { SpecsPanel } from '@/features/specs/components/SpecsPanel';

import { getProductById } from '@/data/products';
import { useCartStore } from '@/store/cartStore';

export default function ProductDetailScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();

  const [selectedTab, setSelectedTab] =
    useState<DetailTab>('description');

  const product = getProductById(id);
  const add = useCartStore((state) => state.add);
```

```
  if (!product) {
    return (
      <Screen>
        <Text className="p-4 text-white">
          Product not found.
        </Text>
      </Screen>
    );
  }

  return (
    <Screen>
      <Stack.Screen options={{ headerShown: false }} />

      <View className="flex-1">
        <ScrollView
          showsVerticalScrollIndicator={false}
          contentContainerStyle={{ flexGrow: 1 }}
        >
          {/* Product hero */}
          <BikeHero product={product} />

          {/* Description / Specifications */}
          <View className="flex-1 rounded-t-3xl bg-[#273246] px-4 pt-5">
            <SegmentedToggle
              value={selectedTab}
              onChange={setSelectedTab}
            />

            <View className="mt-6">
              {selectedTab === 'description' ? (
                <>
                  <Text className="text-sm font-bold text-white">
                    {product.name}
                  </Text>

                  <Text className="mt-3 text-xs leading-5 text-gray-400">
                    {product.description}
                  </Text>
                </>
              ) : (
                <SpecsPanel specs={product.specs} />
```

```
              )}
            </View>
          </View>
        </ScrollView>

        {/* Bottom purchasing bar */}
        <View className="flex-row items-center justify-between bg-[#273246]
px-5 py-4">
          <Text className="text-base text-blue-400">
            ${product.price.toLocaleString('en-US', {
              minimumFractionDigits: 2,
              maximumFractionDigits: 2,
            })}
          </Text>

          <AccentButton
            label="Add to Cart"
            onPress={() => add(product.id)}
          />
        </View>
      </View>
    </Screen>
  );
}
```

This is a functional first version of the product detail screen. A few differences remain compared with Figma:

- The custom back button and product title in the hero are not implemented yet.
- The hero image layout needs device-specific adjustment.
- The bottom purchasing bar needs its final rounded shape and shadows.
- The specification panel may need additional content to match the design exactly.

But importantly, the data, navigation, tab selection, and cart action are connected.

7. Connect the cart

Your existing `cartStore.ts` should be responsible for these actions:

```
type CartState = {
  items: { productId: string; qty: number }[];
```

```
  promoCode: string | null;

  add: (id: string) => void;
  increment: (id: string) => void;
  decrement: (id: string) => void;
  remove: (id: string) => void;
  applyPromo: (code: string) => boolean;
  clear: () => void;
};
```

Assuming you've implemented the Zustand store we discussed earlier, the Add to Cart button is already connected through:

```
const add = useCartStore((state) => state.add);
```

And:

```
<AccentButton
  label="Add to Cart"
  onPress={() => add(product.id)}
/>
```

No additional state management is required in the product detail screen.

One improvement I would eventually make is to provide feedback when an item is added, such as changing the button label momentarily to "Added!" or updating a visible cart badge.

Two small implementation corrections

For `SegmentedToggle.tsx`, move the animation update into an effect instead of updating the shared value during rendering:

```
import { useEffect, useState } from 'react';

// Inside SegmentedToggle:

useEffect(() => {
  position.value = withTiming(
```

```
    value === 'description' ? 0 : 1,
    { duration: 220 }
  );
}, [value, position]);
```

Remove the original `position.value = withTiming(...)` from the component body. Also account for the toggle's horizontal padding in the translation distance: the indicator should move `(width - 8) / 2`, not `width / 2`.

For `BikeHero`, note that it is written to show any product image. If the product is a helmet, you may eventually want to use a different image layout than a full-width bicycle.

8. Final implementation checklist

Development checklist 0/15 complete

1. Finish product listing

- [ ] Featured bicycle image and discount label
- [ ] Background gradient adjustment
- [ ] Product card sizing and typography
- [ ] Category icons and selected gradient
- [ ] Staggered grid layout

2. Product detail

- [ ] BikeHero diagonal background and image
- [ ] Header with back button and title
- [ ] Animated SegmentedToggle
- [ ] Description and SpecsPanel
- [ ] Fixed price and Add to Cart bar

3. Functional verification

- [ ] Tap a product to navigate by ID
- [ ] Test category filtering
- [ ] Test both detail tabs

- [ ] Add a product and verify cart state
- [ ] Check iOS layout and safe areas

My recommendation: Implement `PromoBanner` and the `ProductCard` refinements first, then build the three product detail components in the order shown.

The most important thing is that you now have a clean separation between reusable UI, product-specific presentation, screen navigation, and Zustand state.

At this point, I'd resist adding any more abstractions. Once both screens look like Figma and the cart interactions work, focus on animation quality, responsive sizing, and edge cases. Those will make a bigger difference to the finished take-home than additional architectural complexity.

You said:

[id].tsx
Code

BikeHero.tsx
Code

```
import { useEffect, useState } from 'react';
import { Pressable, Text, View } from 'react-native';
import Animated, {
   useAnimatedStyle,
   useSharedValue,
   withTiming,
} from 'react-native-reanimated';

export type DetailTab = 'description' | 'specification';

type Props = {
  value: DetailTab;
  onChange: (value: DetailTab) => void;
};

const tabs: { value: DetailTab; label: string }[] = [
  { value: 'description', label: 'Description' },
  { value: 'specification', label: 'Specification' },
];

export function SegmentedToggle({ value, onChange }: Props) {
  const [width, setWidth] = useState(0);
```

```
const translateX = useSharedValue(0);

const tabWidth = Math.max(0, (width - 8) / 2);

useEffect(() => {
 translateX.value = withTiming(
  value === 'description' ? 0 : tabWidth,
  { duration: 220 }
 );
}, [value, tabWidth, translateX]);

const indicatorStyle = useAnimatedStyle(() => ({
 transform: [{ translateX: translateX.value }],
}));

return (
 <View
  onLayout={(event) => setWidth(event.nativeEvent.layout.width)}
  className="flex-row rounded-xl bg-[#242C3C] p-1"
 >
  <Animated.View
   pointerEvents="none"
   style={[
    {
     position: 'absolute',
     top: 4,
     bottom: 4,
     left: 4,
     width: tabWidth,
     borderRadius: 8,
     backgroundColor: '#334158',
    },
    indicatorStyle,
   ]}
  />

  {tabs.map((tab) => {
   const selected = value === tab.value;

   return (
    <Pressable
     key={tab.value}
```

```
          accessibilityRole="tab"
          accessibilityState={{ selected }}
          onPress={() => onChange(tab.value)}
          className="min-h-11 flex-1 items-center justify-center px-
2"
        >
          <Text
            className={`text-xs font-semibold ${
              selected ? 'text-[#36B9F4]' : 'text-gray-400'
            }`}
          >
            {tab.label}
          </Text>
        </Pressable>
      );
    })}
  </View>
 );
}


import { Text } from 'react-native';
import { LinearGradient } from 'expo-linear-gradient';
import { PressableScale } from './PressableScale';
import { gradients } from '@/theme/tokens';

type Props = {
  label: string;
  onPress?: () => void;
  className?: string;
};

export function AccentButton({ label, onPress, className = '' }:
Props) {
  return (
    <PressableScale onPress={onPress} className={className}>
    <LinearGradient
      {...gradients.accent}
      style={{
        width: 130,
        height: 36,
        borderRadius: 10,
```

```
            alignItems: 'center',
            justifyContent: 'center',
          }}
          >
          <Text className="text-white text-xs font-semibold">{label}
</Text>
        </LinearGradient>
      </PressableScale>
    );
}
```

Can you verify if the product details are completed based on these code snippets? I think they are but please double check. If that's good we should do the cart screen next. Here we need to start or edit these:

cart.tsx
CartItem.tsx
QuantityStepper.tsx
PromoInput.tsx
CartSummary.tsx
cartStore.ts

Does this track?

ChatGPT said:

Yes, the architecture tracks, and **the cart screen is the right next feature to build**. However, after reviewing your actual `product/[id].tsx`, `BikeHero.tsx`, `SegmentedToggle`, and `AccentButton`, I found one important detail: the product details aren't quite finished yet.

1. Product detail review

Component	Status	Assessment
`SegmentedToggle.tsx`	✅ Ready	Controlled component, correct indicator positioning, and Reanimated transition
`AccentButton.tsx`	✅ Ready	Reusable gradient button with press animation

`BikeHero.tsx`	✅ Ready to integrate	Accepts a `Product` and renders the diagonal gradient behind the image
`product/[id].tsx`	⚠️ Needs integration	Still uses a basic `Image`, description, and purchase button layout
`SpecsPanel.tsx`	Not verified	I haven't seen your current implementation
`cartStore.ts`	Not verified as implemented	The last version you provided only declared its state type

The main issue is that your uploaded `product/[id].tsx` doesn't actually render `BikeHero`, `SegmentedToggle`, or `SpecsPanel` yet. [id].tsx

Your `BikeHero` implementation itself looks reasonable, although you'll want to test the diagonal positioning on different screen widths. BikeHero.tsx

I would finish the integration before moving to the cart, but it should be a relatively small task.

What to change in `product/[id].tsx`

Keep your existing imports for `Screen`, `AccentButton`, `getProductById`, and `useCartStore`. Add:

```
import { useState } from 'react';
import { ScrollView } from 'react-native';

import {
  SegmentedToggle,
  type DetailTab,
} from '@/components/ui/SegmentedToggle';

import { BikeHero } from '@/features/products/components/BikeHero';
import { SpecsPanel } from '@/features/specs/components/SpecsPanel';
```

Then add the selected tab inside your component:

```
const [selectedTab, setSelectedTab] =
  useState<DetailTab>('description');
```

Your existing `getProductById(id)` and `add(product.id)` connections can remain unchanged.

Replace the current screen layout with:

```
return (
  <Screen>
    <Stack.Screen options={{ title: product.name }} />

    <View className="flex-1">
      <ScrollView
        showsVerticalScrollIndicator={false}
        contentContainerStyle={{ flexGrow: 1 }}
      >
        <BikeHero product={product} />

        <View className="flex-1 rounded-t-3xl bg-surface px-4 pt-5">
          <SegmentedToggle
            value={selectedTab}
            onChange={setSelectedTab}
          />

          <View className="mt-6">
            {selectedTab === 'description' ? (
              <>
                <Text className="text-sm font-bold text-white">
                  {product.name}
                </Text>

                <Text className="mt-3 text-xs leading-5 text-gray-400">
                  {product.description}
                </Text>
              </>
            ) : (
              <SpecsPanel specs={product.specs} />
            )}
          </View>
        </View>
      </ScrollView>

      <View className="flex-row items-center justify-between bg-surface px-5
```

```
py-4">
        <Text className="text-base text-blue-400">
          ${product.price.toFixed(2)}
        </Text>

        <AccentButton
          label="Add to Cart"
          onPress={() => add(product.id)}
        />
      </View>
    </View>
  </Screen>
);
```

That completes the primary component connections. The custom Figma header, carousel indicators, and final styling can be polished afterward.

One minor detail: your `AccentButton` is fixed at 130 × 36. That's perfectly acceptable for the Add to Cart button in this particular design.

2. Implement the cart screen

Your proposed six files are exactly what I'd use:

```
app/
  cart.tsx

src/
  features/
    cart/
      components/
        CartItem.tsx
        QuantityStepper.tsx
        PromoInput.tsx
        CartSummary.tsx

  store/
    cartStore.ts
```

I'd keep your existing `src/store/cartStore.ts` location for now, since you're already importing it elsewhere.

The responsibilities will be:

File	Responsibility
`cart.tsx`	Screen composition and connecting components
`CartItem.tsx`	Product image, name, price, quantity
`QuantityStepper.tsx`	Increase and decrease controls
`PromoInput.tsx`	Promo code entry and validation feedback
`CartSummary.tsx`	Subtotal, delivery, discount, total
`cartStore.ts`	Shared cart state and actions

Step 1 — `cartStore.ts`

Start with the state layer because everything else will depend on it.

```
import { create } from 'zustand';
import { getProductById } from '@/data/products';

export type CartItemData = {
  productId: string;
  qty: number;
};

type CartState = {
  items: CartItemData[];
  promoCode: string | null;

  add: (id: string) => void;
  increment: (id: string) => void;
  decrement: (id: string) => void;
  remove: (id: string) => void;
  applyPromo: (code: string) => boolean;
  clear: () => void;
};
```

```
const promoCodes: Record<string, number> = {
  BIKE30: 0.3,
};

export const useCartStore = create<CartState>((set) => ({
  items: [],
  promoCode: null,

  add: (id) => {
    if (!getProductById(id)) return;

    set((state) => {
      const exists = state.items.some(
        (item) => item.productId === id
      );

      return {
        items: exists
          ? state.items.map((item) =>
              item.productId === id
                ? { ...item, qty: item.qty + 1 }
                : item
            )
          : [...state.items, { productId: id, qty: 1 }],
      };
    });
  },

  increment: (id) => {
    set((state) => ({
      items: state.items.map((item) =>
        item.productId === id
          ? { ...item, qty: item.qty + 1 }
          : item
      ),
    }));
  },

  decrement: (id) => {
    set((state) => ({
      items: state.items
        .map((item) =>
```

```
            item.productId === id
              ? { ...item, qty: item.qty - 1 }
              : item
          )
          .filter((item) => item.qty > 0),
      }));
    },

    remove: (id) => {
      set((state) => ({
        items: state.items.filter(
          (item) => item.productId !== id
        ),
      }));
    },

    applyPromo: (code) => {
      const normalized = code.trim().toUpperCase();

      if (!Object.prototype.hasOwnProperty.call(promoCodes, normalized)) {
        return false;
      }

      set({ promoCode: normalized });
      return true;
    },

    clear: () => set({
      items: [],
      promoCode: null,
    }),
  }));

  export function getCartTotals(
    items: CartItemData[],
    promoCode: string | null
  ) {
    const subtotal = items.reduce((total, item) => {
      const product = getProductById(item.productId);

      return total + (product?.price ?? 0) * item.qty;
    }, 0);
```

```
  const discountRate = promoCode
    ? promoCodes[promoCode] ?? 0
    : 0;

  const discount = Math.round(subtotal * discountRate * 100) / 100;
  const delivery = 0;
  const total = Math.round((subtotal - discount + delivery) * 100) / 100;

  return { subtotal, discount, delivery, total };
}
```

This follows the same style you've been writing and keeps the calculations separate from the store updates.

`BIKE30` and free delivery are mock business rules. They reflect the 30% discount and free-delivery presentation in your Figma, but we can adjust the rules if the task specifies something different.

Step 2 — `QuantityStepper.tsx`

The Figma shows a small minus button, quantity indicator, and bright-blue plus button.

```
import { Pressable, Text, View } from 'react-native';
import { Minus, Plus } from 'lucide-react-native';

type Props = {
  quantity: number;
  onIncrement: () => void;
  onDecrement: () => void;
};

export function QuantityStepper({
  quantity,
  onIncrement,
  onDecrement,
}: Props) {
  return (
    <View className="flex-row items-center gap-2">
      <Pressable
        onPress={onDecrement}
```

```
        accessibilityRole="button"
        accessibilityLabel="Decrease quantity"
        className="h-6 w-6 items-center justify-center rounded bg-[#354158]"
      >
        <Minus size={12} color="#FFFFFF" />
      </Pressable>

      <Text className="text-xs text-white">
        {quantity}
      </Text>

      <Pressable
        onPress={onIncrement}
        accessibilityRole="button"
        accessibilityLabel="Increase quantity"
        className="h-6 w-6 items-center justify-center rounded bg-[#3655EA]"
      >
        <Plus size={12} color="#FFFFFF" />
      </Pressable>
    </View>
  );
}
```

Step 3 — `CartItem.tsx`

Each cart item can retrieve its product using the ID stored in Zustand.

This means we're not duplicating product names, prices, or images in the cart state.

```
import { Image, Text, View } from 'react-native';

import { CartItemData } from '@/store/cartStore';
import { getProductById } from '@/data/products';
import { QuantityStepper } from './QuantityStepper';

type Props = {
  item: CartItemData;
  onIncrement: () => void;
  onDecrement: () => void;
};
```

```
export function CartItem({
  item,
  onIncrement,
  onDecrement,
}: Props) {
  const product = getProductById(item.productId);

  if (!product) return null;

  return (
    <View className="flex-row items-center gap-3 border-b border-white/10 py-
4">
      <View className="h-20 w-20 items-center justify-center rounded-xl bg-
[#354158]">
        <Image
          source={product.image}
          resizeMode="contain"
          className="h-16 w-16"
        />
      </View>

      <View className="flex-1">
        <Text
          numberOfLines={2}
          className="text-xs font-semibold text-white"
        >
          {product.name}
        </Text>

        <Text className="mt-2 text-xs text-blue-400">
          ${product.price.toFixed(2)}
        </Text>

        <View className="mt-2 items-end">
          <QuantityStepper
            quantity={item.qty}
            onIncrement={onIncrement}
            onDecrement={onDecrement}
          />
        </View>
      </View>
    </View>
```

```
  );
}
```

Step 4 — `PromoInput.tsx`

This component will own the temporary text input state, but the cart store will own the successfully applied promo code.

```
import { useState } from 'react';
import { Text, TextInput, View } from 'react-native';

import { AccentButton } from '@/components/ui/AccentButton';

type Props = {
  onApply: (code: string) => boolean;
};

export function PromoInput({ onApply }: Props) {
  const [code, setCode] = useState('');
  const [message, setMessage] = useState('');

  function handleApply() {
    const success = onApply(code);

    setMessage(
      success ? 'Promo code applied!' : 'Invalid promo code'
    );
  }

  return (
    <View className="gap-2">
      <View className="flex-row items-center rounded-xl bg-[#20293A] p-1">
        <TextInput
          value={code}
          onChangeText={(text) => {
            setCode(text);
            setMessage('');
          }}
          placeholder="Promo code"
          placeholderTextColor="#8D98AB"
          autoCapitalize="characters"
```

```
          className="h-9 flex-1 px-3 text-xs text-white"
        />

        <AccentButton
          label="Apply"
          onPress={handleApply}
        />
      </View>

      {!!message && (
        <Text className="text-xs text-gray-400">
          {message}
        </Text>
      )}
    </View>
  );
}
```

Later, you might want the `AccentButton` to support different sizes. For now, your existing fixed-width implementation is adequate.

Step 5 — `CartSummary.tsx`

Keep this component purely presentational.

```
import { Text, View } from 'react-native';

type Props = {
  subtotal: number;
  discount: number;
  delivery: number;
  total: number;
};

export function CartSummary({
  subtotal,
  discount,
  delivery,
  total,
}: Props) {
  function Row({
```

```
      label,
      value,
    }: {
      label: string;
      value: string;
    }) {
      return (
        <View className="flex-row justify-between">
          <Text className="text-xs text-gray-300">
            {label}
          </Text>

          <Text className="text-xs text-gray-300">
            {value}
          </Text>
        </View>
      );
    }

    return (
      <View className="gap-3">
        <Row label="Subtotal" value={`$${subtotal.toFixed(2)}`} />
        <Row label="Delivery Fee" value={`$${delivery.toFixed(2)}`} />
        <Row label="Discount" value={`-$${discount.toFixed(2)}`} />

        <View className="mt-2 flex-row justify-between">
          <Text className="text-sm font-semibold text-white">
            Total
          </Text>

          <Text className="text-sm font-bold text-[#36B9F4]">
            ${total.toFixed(2)}
          </Text>
        </View>
      </View>
    );
  }
```

I'd eventually move the small `Row` component outside `CartSummary` so it's not recreated every render. That's a minor cleanup, not a blocker.

Step 6 — Connect everything in `cart.tsx`

This screen should be responsible for composing the components and connecting them to Zustand.

```
import { ScrollView, Text, View } from 'react-native';
import { useRouter } from 'expo-router';

import { Screen } from '@/components/ui/Screen';
import { AccentButton } from '@/components/ui/AccentButton';

import { CartItem } from '@/features/cart/components/CartItem';
import { PromoInput } from '@/features/cart/components/PromoInput';
import { CartSummary } from '@/features/cart/components/CartSummary';

import {
  useCartStore,
  getCartTotals,
} from '@/store/cartStore';

export default function CartScreen() {
  const router = useRouter();

  const items = useCartStore((state) => state.items);
  const promoCode = useCartStore((state) => state.promoCode);
  const increment = useCartStore((state) => state.increment);
  const decrement = useCartStore((state) => state.decrement);
  const applyPromo = useCartStore((state) => state.applyPromo);

  const totals = getCartTotals(items, promoCode);

  return (
    <Screen>
      <View className="flex-1 px-4">
        <Text className="py-4 text-lg font-bold text-white">
          My Shopping Cart
        </Text>

        <ScrollView
          showsVerticalScrollIndicator={false}
          contentContainerStyle={{ flexGrow: 1, paddingBottom: 24 }}
        >
```

```
        {items.length === 0 ? (
          <View className="flex-1 items-center justify-center">
            <Text className="text-sm text-gray-400">
              Your cart is empty.
            </Text>
          </View>
        ) : (
          <>
            {items.map((item) => (
              <CartItem
                key={item.productId}
                item={item}
                onIncrement={() => increment(item.productId)}
                onDecrement={() => decrement(item.productId)}
              />
            ))}

            <Text className="mt-8 mb-4 text-center text-xs text-gray-400">
              Your cart qualifies for free shipping
            </Text>

            <PromoInput onApply={applyPromo} />

            <View className="mt-6">
              <CartSummary {...totals} />
            </View>
          </>
        )}
      </ScrollView>

      {items.length > 0 && (
        <View className="items-center py-4">
          <AccentButton
            label="Checkout"
            onPress={() => {
              // Connect to checkout confirmation next.
            }}
          />
        </View>
      )}
    </View>
  </Screen>
```

```
  );
}
```

The `router` import and variable aren't needed until you wire the next route, so you can remove those for now. The checkout button above is deliberately a placeholder: it shouldn't be considered complete until it performs the slide-to-confirm interaction required by Figma.

3. What is still missing?

The remaining connection is `SlideToConfirm.tsx`, which you originally included in your UI architecture.

Your Figma specifically requests a left-to-right sliding checkout interaction, rather than just a conventional button.

I would build that after the cart layout, ideally using Reanimated and Gesture Handler, then replace the temporary checkout button.

We should also add the custom header/back button, finish the product detail styling, and confirm that the cart is reachable through navigation.

The important behaviors to test before calling the cart complete are changing quantities, removing the final unit, applying an invalid promo code without losing a previously valid discount, updating totals, and navigating back to the product screen without losing cart state.

So yes: your proposed structure is right. Finish the small product-detail integration first, then implement these six cart files in order. After that, the main remaining work is the slide-to-confirm checkout interaction and Figma-level visual polish.

You said:

```
import { getProductById } from '@/data/products';
import { create } from 'zustand';

type CartItem = {
  productId: string;
  qty: number;
};

type CartState = {
```

```
  items: CartItem[];
  promoCode: string | null;

  add: (id: string) => void;
  increment: (id: string) => void;
  decrement: (id: string) => void;
  remove: (id: string) => void;
  applyPromo: (code: string) => boolean;
  clear: () => void;
};

const promoCodes: Record<string, number> = {
  BIKE30: 0.30,
};

const bumpQty = (items: CartItem[], id: string) => {
    return items.map((item) =>
        item.productId === id
            ? { ...item, qty: item.qty + 1 }
            : item
    );
};

export const useCartStore = create<CartState>((set) => ({
  items: [],
  promoCode: null,

  add: (id) => {
    if (!getProductById(id)) return;

    set((state) => {
      const existing = state.items.find(
        (item) => item.productId === id
      );

      if (existing) {
        return { items: bumpQty(state.items, id) };
      }

      return {
        items: [...state.items, { productId: id, qty: 1 }],
      };
```

```
    });
  },

  increment: (id) => {
    if (!getProductById(id)) return;
    set((state) => ({ items: bumpQty(state.items, id ) }));
  },

  decrement: (id) => {
    set((state) => ({
      items: state.items
        .map((item) =>
          item.productId === id
            ? { ...item, qty: item.qty - 1 }
            : item
        )
        .filter((item) => item.qty > 0),
    }));
  },

  remove: (id) => {
    set((state) => ({
      items: state.items.filter(
        (item) => item.productId !== id
      ),
    }));
  },

  applyPromo: (code) => {
    const normalized = code.trim().toUpperCase();

    if (!Object.prototype.hasOwnProperty.call(promoCodes,
normalized)) {
      return false;
    }

    set({ promoCode: normalized });
    return true;
  },

  clear: () => set({
    items: [],
```

```
    promoCode: null,
  }),
}));

export function getCartTotals(
  items: CartItem[],
  promoCode: string | null
) {
  const subtotal = items.reduce((total, item) => {
    const product = getProductById(item.productId);

    return total + (product?.price ?? 0) * item.qty;
  }, 0);

  const discountRate = promoCode
    ? promoCodes[promoCode] ?? 0
    : 0;

  const discount = subtotal * discountRate;
  const total = subtotal - discount;

  return {
    subtotal,
    discount,
    total,
  };
}
```

Would you switch off this existing cartStore?

ChatGPT said:

No, **I would keep your existing** `cartStore.ts`. In fact, I prefer your implementation over the one I suggested earlier.

You've already made a good improvement by extracting `bumpQty()` into a reusable helper. Your store is concise, readable, and follows Zustand's intended pattern.

I wouldn't rewrite it. I'd make three minor adjustments.

1. Export `CartItem`

Currently you have:

```
type CartItem = {
  productId: string;
  qty: number;
};
```

Change it to:

```
export type CartItem = {
  productId: string;
  qty: number;
};
```

This allows `CartItem.tsx` to reuse the existing type:

```
import type { CartItem as CartItemData } from '@/store/cartStore';
```

That avoids creating duplicate type definitions.

2. Small improvement to `increment()`

Your current implementation:

```
increment: (id) => {
  if (!getProductById(id)) return;
  set((state) => ({ items: bumpQty(state.items, id) }));
},
```

This works, but the product lookup is unnecessary here because `increment()` should only operate on products already in the cart.

I'd simplify it to:

```
increment: (id) => {
  set((state) => ({
    items: bumpQty(state.items, id),
  }));
},
```

If the ID isn't in the cart, `bumpQty()` leaves quantities unchanged.

The lookup in `add()` should stay, however, since it prevents invalid products from being inserted into the cart.

3. Add delivery and currency rounding to `getCartTotals()`

Your calculations are correct, but the Figma's cart summary includes a delivery fee.

I would extend the function slightly:

```
export function getCartTotals(
  items: CartItem[],
  promoCode: string | null
) {
  const subtotal = items.reduce((total, item) => {
    const product = getProductById(item.productId);

    return total + (product?.price ?? 0) * item.qty;
  }, 0);

  const discountRate = promoCode
    ? promoCodes[promoCode] ?? 0
    : 0;

  const discount =
    Math.round(subtotal * discountRate * 100) / 100;

  const delivery = 0;

  const total =
    Math.round((subtotal - discount + delivery) * 100) / 100;

  return {
```

```
    subtotal,
    discount,
    delivery,
    total,
  };
}
```

The Figma displays free shipping, so setting `delivery = 0` is reasonable for this mock implementation.

Rounding the discount and total to two decimal places helps avoid floating-point precision issues with currency. For a production payment system, storing monetary amounts in integer cents would be more robust, but that would be unnecessary complexity for this UI task.

4. What you've already done well

Your store has several good qualities:

Feature	Assessment
TypeScript types	Good
Zustand implementation	Good
Immutable state updates	Good
Reusable `bumpQty()` helper	Good
Product validation in `add()`	Good
Quantity decrement and automatic removal	Good
Promo code normalization	Good
Invalid promo code handling	Good
Pure `getCartTotals()` calculation	Good
Cart reset functionality	Good

One subtle detail: your `applyPromo()` correctly preserves an existing valid promo code if the user submits an invalid one. That's a sensible behavior.

And your store only saves IDs and quantities, rather than duplicating product objects. This is particularly appropriate with your local mock catalog.

5. One potential improvement for later

Your `getCartTotals()` is a pure function, which is excellent for testing.

For example:

```
const result = getCartTotals(
  [{ productId: 'peugeot-lr01', qty: 2 }],
  'BIKE30'
);

console.log(result);
```

Given your current catalog, this should produce:

```
{
  subtotal: 3999.98,
  discount: 1199.99,
  delivery: 0,
  total: 2799.99
}
```

You can unit-test this independently from React Native or Zustand.

My recommendation

Keep your `cartStore.ts` almost exactly as it is.

Make the three small changes above, then move directly into implementing:

1. `QuantityStepper.tsx`
2. `CartItem.tsx`
3. `PromoInput.tsx`
4. `CartSummary.tsx`
5. `cart.tsx`

Your existing store already supports all five components.

I also wouldn't introduce Zustand persistence, additional selectors, or more state abstractions at this point. Those aren't necessary to reproduce the Figma.

You're at the stage where getting the cart UI and checkout interactions working is more valuable than refining an already solid store.

You said:

```
import '@/global.css';

import { DarkTheme, Stack, ThemeProvider } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';

import { AnimatedSplashOverlay } from '@/components/animated-
icon';

SplashScreen.preventAutoHideAsync();

export default function RootLayout() {
 return (
  <ThemeProvider value={DarkTheme}>
   <Stack
    screenOptions={{
     headerStyle: { backgroundColor: '#242C3C' },
     headerTintColor: '#FFFFFF',
     contentStyle: { backgroundColor: '#242C3C' },
    }}
   >
    <Stack.Screen
     name="(tabs)"
     options={{ headerShown: false }}
    />

    <Stack.Screen
     name="product/[id]"
     options={{ title: 'Product details' }}
    />
   </Stack>

   <AnimatedSplashOverlay />
```

```
    </ThemeProvider>
  );
}
```

What do you suggest for SlideToConfirm? While we're at this let's also implement our animations in the layout code attached

ChatGPT said:

I'd use **React Native Gesture Handler + Reanimated** for `SlideToConfirm.tsx`. You already use Reanimated for `PressableScale` and `SegmentedToggle`, so this fits your existing architecture.

For the navigation animations, Expo Router's native `Stack` already supports the transitions shown in Figma. We don't need to build custom screen animations.

The two motion requirements are:

Interaction	Implementation
Product listing → Product detail	Slide in from the right
Product detail → Cart	Slide in from the right
Checkout confirmation	Drag the arrow from left to right
Neumorphic buttons	Existing `PressableScale`
Description / Specification	Existing `SegmentedToggle`

One important distinction: the Figma says checkout should slide from left to right. I'd interpret that as the **checkout confirmation control**, not as a requirement to reverse the usual screen navigation.

1. Update your root `_layout.tsx`

Your current layout is already correct structurally. I'd mainly add animation options and register the cart route explicitly.

```
import '@/global.css';
```

```
import { DarkTheme, Stack, ThemeProvider } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import * as SplashScreen from 'expo-splash-screen';

import { AnimatedSplashOverlay } from '@/components/animated-icon';

SplashScreen.preventAutoHideAsync();

export default function RootLayout() {
  return (
    <ThemeProvider value={DarkTheme}>
      <Stack
        screenOptions={{
          headerStyle: { backgroundColor: '#242C3C' },
          headerTintColor: '#FFFFFF',
          contentStyle: { backgroundColor: '#242C3C' },

          animation: 'slide_from_right',
          animationDuration: 300,
          gestureEnabled: true,
          fullScreenGestureEnabled: true,
        }}
      >
        <Stack.Screen
          name="(tabs)"
          options={{ headerShown: false }}
        />

        <Stack.Screen
          name="product/[id]"
          options={{
            title: 'Product details',
          }}
        />

        <Stack.Screen
          name="cart"
          options={{
            title: 'My Shopping Cart',
          }}
        />
      </Stack>
```

```
      <StatusBar style="light" />
      <AnimatedSplashOverlay />
    </ThemeProvider>
  );
}
```

This gives you the usual iOS navigation behavior:

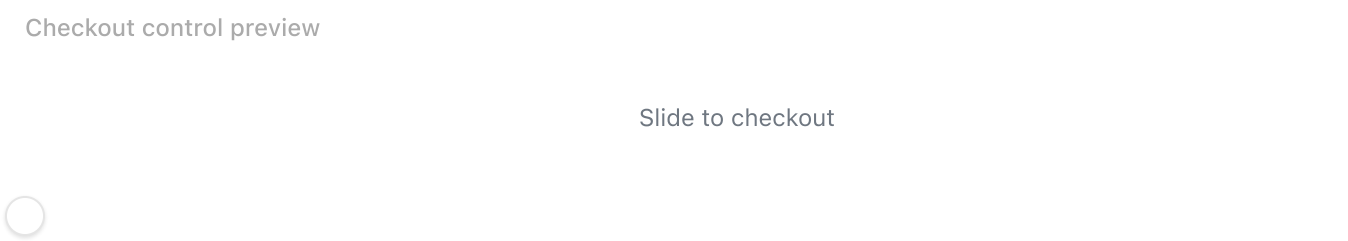
- New screens enter from the right.
- Previous screens move back into view when navigating backward.
- Users can swipe back on supported iOS screens.
- Your dark navigation theme remains consistent.

Note that native-stack animation duration customization is limited by platform and animation type, so treat `300` as a preference rather than a guarantee of identical timing on iOS and Android.

One important routing detail: If your cart is located inside the `(tabs)` group rather than at `app/cart.tsx`, don't register a separate root `cart` screen. Configure it in the corresponding tab layout instead.

2. Implement `SlideToConfirm.tsx`

Here's the interaction I would build:



Interactive concept preview. The React Native implementation below makes the arrow itself draggable.

The thumb should:

1. Follow the user's finger horizontally.
2. Stay inside the track.
3. Snap back if released before the completion threshold.

4. Animate to the end if the threshold is reached.
5. Call `onConfirm()` exactly once.

We'll keep it independent of Zustand or checkout navigation, just like your other reusable UI components.

First, ensure Gesture Handler is installed:

```
npx expo install react-native-gesture-handler
```

Reanimated is already part of your project.

`src/components/ui/SlideToConfirm.tsx`

```
import { useState } from 'react';
import { Text, View } from 'react-native';
import { LinearGradient } from 'expo-linear-gradient';
import { ChevronRight } from 'lucide-react-native';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';

import Animated, {
  runOnJS,
  useAnimatedStyle,
  useSharedValue,
  withSpring,
  withTiming,
} from 'react-native-reanimated';

import { gradients } from '@/theme/tokens';

type Props = {
  onConfirm: () => void;
  disabled?: boolean;
};

const THUMB_SIZE = 42;
const PADDING = 5;
const THRESHOLD = 0.85;

export function SlideToConfirm({
```

```
    onConfirm,
    disabled = false,
  }: Props) {
    const [width, setWidth] = useState(0);
    const [completed, setCompleted] = useState(false);

    const translateX = useSharedValue(0);

    const maxTranslate = Math.max(
      0,
      width - THUMB_SIZE - PADDING * 2
    );

    function handleConfirm() {
      setCompleted(true);
      onConfirm();
    }

    const gesture = Gesture.Pan()
      .enabled(!disabled && !completed && maxTranslate > 0)
      .onUpdate((event) => {
        translateX.value = Math.min(
          Math.max(event.translationX, 0),
          maxTranslate
        );
      })
      .onEnd(() => {
        if (translateX.value >= maxTranslate * THRESHOLD) {
          translateX.value = withTiming(maxTranslate, {
            duration: 180,
          });

          runOnJS(handleConfirm)();
        } else {
          translateX.value = withSpring(0, {
            damping: 16,
            stiffness: 180,
          });
        }
      });

    const animatedStyle = useAnimatedStyle(() => ({
```

```
      transform: [{ translateX: translateX.value }],
    }));

    return (
      <View
        onLayout={(event) =>
          setWidth(event.nativeEvent.layout.width)
        }
        className="h-[52px] justify-center overflow-hidden rounded-xl bg-
[#20293A] p-[5px]"
      >
        <Text className="absolute self-center text-xs text-gray-400">
          {completed ? 'Confirmed!' : 'Slide to checkout'}
        </Text>

        <GestureDetector gesture={gesture}>
          <Animated.View
            style={[
              {
                width: THUMB_SIZE,
                height: THUMB_SIZE,
                borderRadius: 10,
              },
              animatedStyle,
            ]}
          >
            <LinearGradient
              {...gradients.accent}
              style={{
                flex: 1,
                borderRadius: 10,
                alignItems: 'center',
                justifyContent: 'center',
              }}
            >
              <ChevronRight size={22} color="#FFFFFF" />
            </LinearGradient>
          </Animated.View>
        </GestureDetector>
      </View>
    );
  }
```

This follows your existing coding conventions and uses the same accent gradient as `AccentButton`.

The confirmation callback runs on the JavaScript thread, while the actual drag animation runs on the UI thread.

One consideration: after confirmation, this control stays completed while mounted. That's desirable for preventing accidental repeated checkout actions. If you need to reset it for another checkout attempt without unmounting, we can add a `resetKey` prop later.

3. Add Gesture Handler to your root layout

Because `SlideToConfirm` uses Gesture Handler, wrap the navigation tree with `GestureHandlerRootView`.

Here is the complete recommended `_layout.tsx`:

```
import '@/global.css';

import { DarkTheme, Stack, ThemeProvider } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import * as SplashScreen from 'expo-splash-screen';
import { GestureHandlerRootView } from 'react-native-gesture-handler';

import { AnimatedSplashOverlay } from '@/components/animated-icon';

SplashScreen.preventAutoHideAsync();

export default function RootLayout() {
  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <ThemeProvider value={DarkTheme}>
        <Stack
          screenOptions={{
            headerStyle: {
              backgroundColor: '#242C3C',
            },
            headerTintColor: '#FFFFFF',
            contentStyle: {
              backgroundColor: '#242C3C',
```

```
            },

            animation: 'slide_from_right',
            gestureEnabled: true,
            fullScreenGestureEnabled: true,
          }}
        >
          <Stack.Screen
            name="(tabs)"
            options={{ headerShown: false }}
          />

          <Stack.Screen
            name="product/[id]"
            options={{
              title: 'Product details',
            }}
          />

          <Stack.Screen
            name="cart"
            options={{
              title: 'My Shopping Cart',
            }}
          />
        </Stack>

        <StatusBar style="light" />
        <AnimatedSplashOverlay />
      </ThemeProvider>
    </GestureHandlerRootView>
  );
}
```

I've omitted `animationDuration` here because the native-stack right-slide animation generally uses the platform's transition timing, and that option does not customize every animation type.

Your existing `AnimatedSplashOverlay` remains untouched.

4. Connect the checkout slider to `cart.tsx`

Replace the temporary checkout button in your cart screen with:

```
import { Alert, View } from 'react-native';
import { SlideToConfirm } from '@/components/ui/SlideToConfirm';
```

Inside `CartScreen`:

```
function handleCheckout() {
  Alert.alert(
    'Checkout confirmed',
    'The checkout animation is complete. No payment has been processed.'
  );
}
```

Then, near the bottom of the component:

```
{items.length > 0 && (
  <View className="px-4 py-4">
    <SlideToConfirm onConfirm={handleCheckout} />
  </View>
)}
```

This allows you to demonstrate the requested interaction without implementing an actual payment system.

Later, if the take-home expects an order-completion state, we can connect `handleCheckout` to that flow.

5. Make sure the cart is accessible

One thing I'd verify in your current project: **how does the user navigate from the product detail screen to the cart?**

Your product detail screen currently calls:

```
onPress={() => add(product.id)}
```

That updates Zustand, but doesn't necessarily navigate anywhere.

You could leave Add to Cart as a state-only action and provide a shopping-cart icon in the header that navigates to `/cart`.

For example:

```
import { useRouter } from 'expo-router';

const router = useRouter();

// When the user taps the cart icon:
router.push('/cart');
```

That's preferable to automatically navigating away after every Add to Cart press, because users may want to continue browsing.

Also, since your screenshot currently shows Expo's default Home/Explore tab bar, I'd eventually replace that with your five-icon custom bottom navigation from Figma.

6. Recommended testing sequence

Once these changes are in place, test the following:

- Open the listing screen and select a bicycle.
- Verify the detail screen slides in from the right.
- Confirm that the iOS back gesture works.
- Add the bicycle to the cart.
- Navigate to the cart and confirm the product appears.
- Increase the product quantity and verify the totals update.
- Apply `BIKE30` and verify the discount.
- Drag the checkout arrow halfway and release; it should return to the beginning.
- Drag it beyond 85%; it should complete and trigger the confirmation callback.
- Verify checkout cannot be triggered repeatedly.

One final recommendation: Your `SlideToConfirm` should remain in `components/ui/`, not `features/cart/`, because it has no knowledge of shopping carts. That's consistent with how you've built `PressableScale`, `AccentButton`, and `SegmentedToggle`.

At this point, your architecture supports the main application flow. The remaining work is primarily visual refinement, navigation polish, and validating the interactions against the Figma.

You said:

I think what's left is the API call. The API fetched info should be in SpecsPanel.tsx and info populating the project page here. Is this the right approach?

```
import { Stack, useLocalSearchParams, useRouter } from 'expo-
router';
import { ScrollView, Text, View } from 'react-native';

import { AccentButton } from '@/components/ui/AccentButton';
import { Screen } from '@/components/ui/Screen';
import { DetailTab, SegmentedToggle } from
'@/components/ui/SegmentedToggle';
import { getProductById } from '@/data/products';
import { BikeHero } from
'@/features/products/components/BikeHero';
import { SpecsPanel } from
'@/features/specs/components/SpecsPanel';
import { useCartStore } from '@/store/cartStore';
import { useState } from 'react';


export default function ProductDetailScreen() {
  const router = useRouter();
  const [selectedTab, setSelectedTab] =
    useState<DetailTab>('description');

  const { id } = useLocalSearchParams<{ id: string }>();

  const product = getProductById(id);
  const add = useCartStore((state) => state.add);

  if (!product) {
```

```
    return (
      <Screen edges={['bottom']} animate>
        <Text className='p-4 text-white'>
          Product not found.
        </Text>
      </Screen>
    );
  }

  return (
    <Screen edges={['bottom']} animate>
     <Stack.Screen options={{ title: product.name }} />

     <View className="flex-1 p-4">
      <ScrollView
        showsVerticalScrollIndicator={false}
        contentContainerStyle={{ flexGrow: 1 }}
      >
        <BikeHero product={product}/>

        <View className="flex-1 rounded-t-3xl bg-surface px-4
pt-5">
          <SegmentedToggle
            value={selectedTab}
            onChange={setSelectedTab}
          />

          <View className="mt-6">
            {selectedTab === 'description' ? (
            <>
              <Text className="text-sm font-bold text-white">
                {product.name}
              </Text>

              <Text className="mt-3 text-xs leading-5 text-gray-
400">
                {product.description}
              </Text>
              </>
              ) : (
              <SpecsPanel specs={product.specs} />
              )}
```

```
            </View>
            </View>
          </ScrollView>

          <View className="mt-auto flex-row items-center justify-
between">
            <Text className="text-lg text-blue-400">
              ${product.price.toFixed(2)}
            </Text>

            <AccentButton
              label="Add to Cart"
              onPress={() => { add(product.id); router.push('/cart'); }}
            />
          </View>
        </View>
      </Screen>
    );
}
```

ChatGPT said:

Yes, **you're approaching this correctly**, but I would make one architectural adjustment:

The API request should not happen inside `SpecsPanel.tsx`. It should happen in the product detail screen, or ideally in a dedicated hook that the product detail screen uses.

`SpecsPanel` should remain a presentational component that receives data through props, just as you've already designed it.

This keeps your components modular and makes the API functionality easier to test.

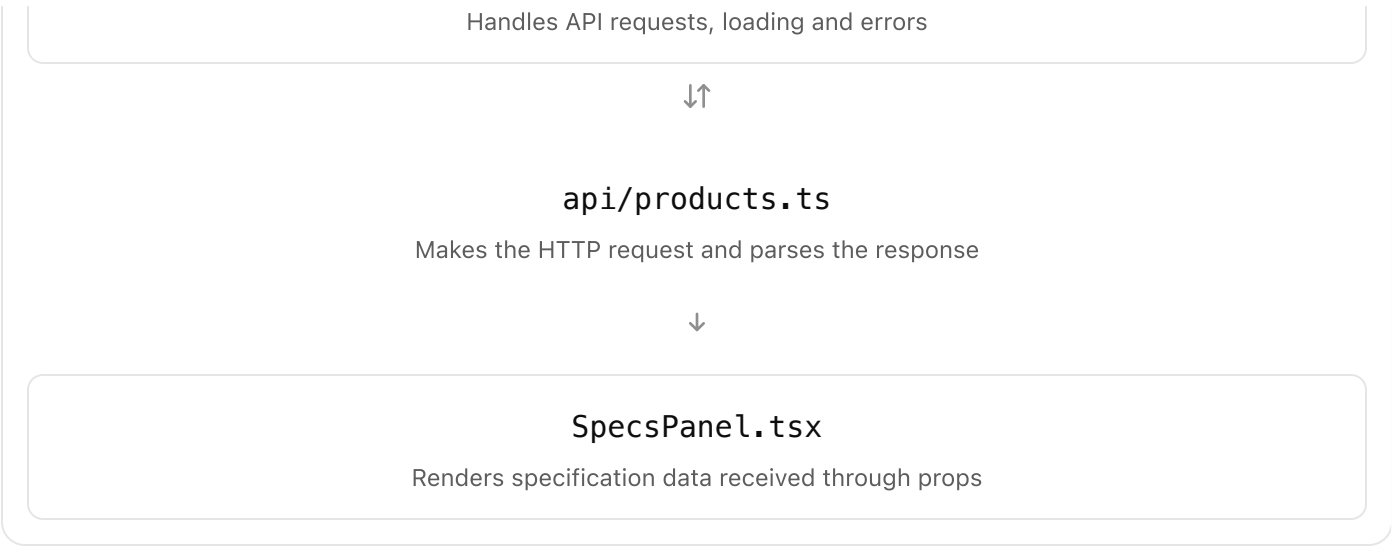
1. Recommended data flow

`app/product/[id].tsx`

Reads product ID, coordinates data and screen state

↓

`useProductSpecs(id)`



This architecture is slightly more involved than calling `fetch()` directly in `SpecsPanel`, but gives you a clean separation of responsibilities.

2. Suggested additions to your existing structure

```
src/
  features/
    specs/
      components/
        SpecsPanel.tsx
      hooks/
        useProductSpecs.ts
      api/
        products.ts
      types.ts
```

You don't need to change anything about your existing `Product` model, `getProductById()`, or cart store to add this.

The main question is what the external API returns.

For the examples below, I'll assume it returns product specifications in this format:

```
{
  "specs": [
    { "label": "Frame", "value": "Lugged steel" },
    { "label": "Drivetrain", "value": "16-speed Shimano Claris" }
```

```
  ]
}
```

This is an example response, not a verified API contract.

3. Create the API function

`src/features/specs/api/products.ts`

```
import type { Product } from '@/types/product';

type ProductSpecs = Product['specs'];

export async function fetchProductSpecs(
  id: string,
  signal?: AbortSignal
): Promise<ProductSpecs> {
  const response = await fetch(
    `https://YOUR_API_URL/products/${encodeURIComponent(id)}/specs`,
    { signal }
  );

  if (!response.ok) {
    throw new Error(`Failed to fetch specs: ${response.status}`);
  }

  const data = await response.json();

  return data.specs;
}
```

You'll need to replace the URL and response parsing with the actual API specification. For production code, I would also validate the response before using it.

4. Create `useProductSpecs.ts`

This hook handles the API lifecycle so the screen doesn't have to.

```
import { useEffect, useState } from 'react';
```

```
import type { Product } from '@/types/product';
import { fetchProductSpecs } from '../api/products';

type ProductSpecs = Product['specs'];

export function useProductSpecs(id: string) {
  const [specs, setSpecs] = useState<ProductSpecs>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const controller = new AbortController();

    setSpecs([]);
    setLoading(true);
    setError(null);

    fetchProductSpecs(id, controller.signal)
      .then((data) => {
        if (!controller.signal.aborted) {
          setSpecs(data);
        }
      })
      .catch((err) => {
        if (!controller.signal.aborted) {
          setError(
            err instanceof Error ? err.message : 'Something went wrong'
          );
        }
      })
      .finally(() => {
        if (!controller.signal.aborted) {
          setLoading(false);
        }
      });

    return () => controller.abort();
  }, [id]);

  return { specs, loading, error };
}
```

The `AbortController` prevents a previous product's request from updating the screen if the user quickly navigates to a different product.

5. Connect the API to your product detail screen

I would keep almost all of your existing code.

First, import the hook:

```
import { ActivityIndicator } from 'react-native';
import { useProductSpecs } from '@/features/specs/hooks/useProductSpecs';
```

Then, inside `ProductDetailScreen` :

```
const { id } = useLocalSearchParams<{ id: string }>();

const product = getProductById(id);

const {
  specs,
  loading,
  error,
} = useProductSpecs(id);
```

Now replace your existing specification branch with:

```
{selectedTab === 'description' ? (
  <>
    <Text className="text-sm font-bold text-white">
      {product.name}
    </Text>

    <Text className="mt-3 text-xs leading-5 text-gray-400">
      {product.description}
    </Text>
  </>
) : loading ? (
  <ActivityIndicator color="#36B9F4" />
) : error ? (
```

```
  <Text className="text-xs text-red-400">
    Unable to load specifications.
  </Text>
) : (
  <SpecsPanel specs={specs} />
)}
```

That's all the UI connection requires.

Your `SpecsPanel` stays exactly as it is.

One consideration: if you want to display your existing mock specifications when the API is unavailable, you could use `product.specs` as fallback data. I would only do that if the task permits fallback data, and clearly distinguish it from successfully loaded API information.

6. A few observations about your current screen

Your product detail screen is now structured well. I'd make three small improvements.

A. Your hook must run before the conditional return.

Even if `getProductById(id)` returns `undefined`, don't place `useProductSpecs()` after your `if (!product)` return. Hooks must be called in a consistent order.

If you need to avoid fetching for an invalid product, add an `enabled` argument to the hook:

```
const { specs, loading, error } =
  useProductSpecs(id, !!product);
```

And only fetch when `enabled` is true.

B. Your Add to Cart behavior is valid.

```
onPress={() => {
  add(product.id);
  router.push('/cart');
}}
```

This is a perfectly reasonable choice. It adds the product and takes the user to checkout.

You don't need additional Zustand logic for this.

C. Your product detail screen already has the correct component boundaries.

`BikeHero`, `SegmentedToggle`, `SpecsPanel`, and `AccentButton` are all connected through the parent screen. That's how I'd structure it.

7. One thing to clarify before implementing the actual API

What is the API endpoint or documentation provided for the take-home task?

The implementation depends significantly on whether the API:

- Returns an entire product object (name, price, description, specs, images).
- Returns only specifications.
- Requires authentication or an API key.
- Uses your own product IDs or different identifiers.

If it returns complete product information, I would likely use the API to populate the whole product detail screen, not just `SpecsPanel`, while keeping the local catalog for the initial listing if that's what the assignment expects.

If you share the API documentation or endpoint, I can adapt the code to the actual response structure rather than introducing assumptions about the data.

You said:

Lets code this with a weather API as it relates to biking

Worked for 39s

ChatGPT said:

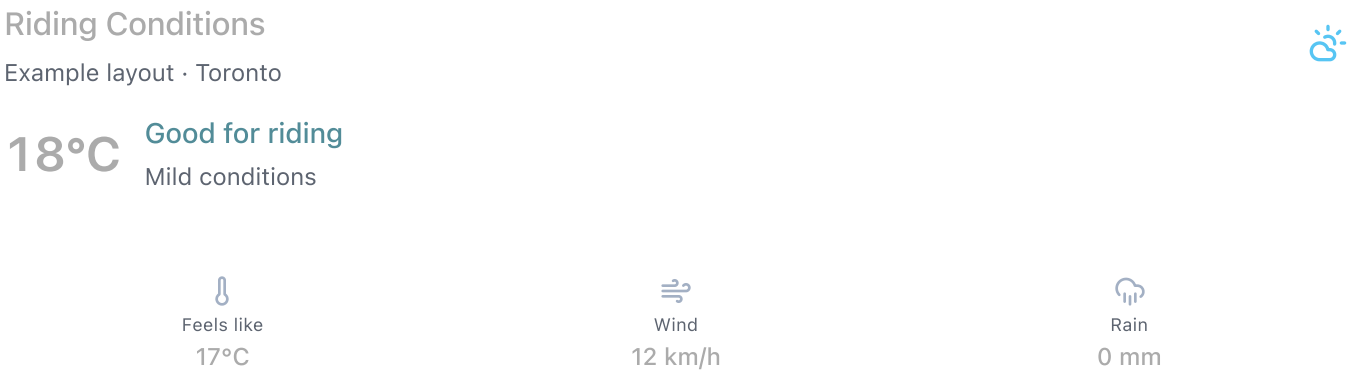
Let's use the **Open-Meteo Weather API** to show whether the weather is suitable for riding the bicycle the user is viewing.

This is a good addition to your Liftoff take-home because it demonstrates an actual HTTP request, asynchronous state handling, TypeScript modeling, and data-driven UI, rather than simply displaying mock product specifications.

Open-Meteo provides current temperature, wind speed, precipitation, and weather conditions without requiring an API key for its non-commercial API. GitHub

1. What we're building

I'd add a small "Riding Conditions" panel to the **Description** section of `product/[id].tsx`, while keeping your existing `SpecsPanel` for the bicycle's actual specifications.



Illustrative UI values only. The actual app will display the response from Open-Meteo.

We'll implement it with these files:

```
src/
  features/
    weather/
      api/
        weather.ts
      hooks/
        useRidingWeather.ts
      components/
        RidingWeatherPanel.tsx
      types.ts

app/
  product/
    [id].tsx
```

I would keep the weather feature separate from `specs/`, since weather is contextual information about riding, not a specification of the bicycle.

2. Create `types.ts`

`src/features/weather/types.ts`

```
export type RidingWeather = {
  temperature: number;
  feelsLike: number;
  windSpeed: number;
  precipitation: number;
  weatherCode: number;
  time: string;
};

export type RidingAssessment = {
  label: string;
  description: string;
  status: 'good' | 'fair' | 'poor';
};
```

The weather information is independent of the product, so we don't need to modify your existing `Product` model.

3. Create the API function

`src/features/weather/api/weather.ts`

We'll use Toronto as an example location for the take-home. Later, you could allow users to select a city or use device location with permission.

```
import type { RidingWeather } from '../types';

type WeatherResponse = {
  current: {
    time: string;
    temperature_2m: number;
    apparent_temperature: number;
    wind_speed_10m: number;
    precipitation: number;
    weather_code: number;
  };
```

```
};

export async function fetchRidingWeather(
  signal?: AbortSignal
): Promise<RidingWeather> {
  const params = new URLSearchParams({
    latitude: '43.6532',
    longitude: '-79.3832',
    current: [
      'temperature_2m',
      'apparent_temperature',
      'wind_speed_10m',
      'precipitation',
      'weather_code',
    ].join(','),
    temperature_unit: 'celsius',
    wind_speed_unit: 'kmh',
    precipitation_unit: 'mm',
    timezone: 'America/Toronto',
  });

  const response = await fetch(
    `https://api.open-meteo.com/v1/forecast?${params.toString()}`,
    { signal }
  );

  if (!response.ok) {
    throw new Error(`Weather request failed: ${response.status}`);
  }

  const data: WeatherResponse = await response.json();
  const current = data.current;

  if (
    !current ||
    ![
      current.temperature_2m,
      current.apparent_temperature,
      current.wind_speed_10m,
      current.precipitation,
      current.weather_code,
    ].every((value) => Number.isFinite(value))
```

```
  ) {
    throw new Error('Invalid weather response');
  }

  return {
    temperature: current.temperature_2m,
    feelsLike: current.apparent_temperature,
    windSpeed: current.wind_speed_10m,
    precipitation: current.precipitation,
    weatherCode: current.weather_code,
    time: current.time,
  };
}
```

The endpoint and requested current-condition fields match the documented Open-Meteo forecast API. GitHub

Using a fixed location for now makes testing predictable and avoids implementing permissions before they're needed.

4. Create a riding-condition assessment

This is where we turn raw weather data into something useful for cyclists.

I'd add this to `weather.ts`, below the API function:

```
import type {
  RidingAssessment,
} from '../types';

export function assessRidingWeather(
  weather: RidingWeather
): RidingAssessment {
  const {
    temperature,
    windSpeed,
    precipitation,
    weatherCode,
  } = weather;

  const severeWeather = [
```

```
      65, 66, 67,
      75, 77,
      82, 85, 86,
      95, 96, 99,
    ].includes(weatherCode);

    if (
      severeWeather ||
      precipitation > 2 ||
      windSpeed > 35 ||
      temperature < 0 ||
      temperature > 35
    ) {
      return {
        label: 'Poor for riding',
        description: 'Conditions may make cycling difficult or unsafe.',
        status: 'poor',
      };
    }

    if (
      precipitation > 0 ||
      windSpeed > 20 ||
      temperature < 10 ||
      temperature > 30
    ) {
      return {
        label: 'Fair for riding',
        description: 'Consider the conditions before heading out.',
        status: 'fair',
      };
    }

    return {
      label: 'Good for riding',
      description: 'Weather conditions look comfortable for cycling.',
      status: 'good',
    };
  }
```

These thresholds are deliberately simple sample rules for the UI demonstration, **not an authoritative cycling safety assessment**. The assessment only considers a few current observations; it does not account for traffic, road conditions, visibility, or changing forecasts. We can improve the logic later.

5. Create `useRidingWeather.ts`

The hook handles the API request and tracks loading, error, and success states.

`src/features/weather/hooks/useRidingWeather.ts`

```
import { useEffect, useState } from 'react';

import { fetchRidingWeather } from '../api/weather';
import type { RidingWeather } from '../types';

export function useRidingWeather() {
  const [weather, setWeather] = useState<RidingWeather | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const controller = new AbortController();

    async function loadWeather() {
      try {
        setLoading(true);
        setError(null);

        const data = await fetchRidingWeather(controller.signal);

        if (!controller.signal.aborted) {
          setWeather(data);
        }
      } catch (err) {
        if (!controller.signal.aborted) {
          setError('Unable to load weather conditions.');
        }
      } finally {
        if (!controller.signal.aborted) {
          setLoading(false);
```

```
        }
      }
    }

    loadWeather();

    return () => controller.abort();
  }, []);

  return { weather, loading, error };
}
```

This matches the same React patterns you've been using. No React Query or additional state library is necessary for a single API request.

6. Create `RidingWeatherPanel.tsx`

This component will be responsible for displaying the weather. No fetching logic is needed inside it.

`src/features/weather/components/RidingWeatherPanel.tsx`

```
import { ActivityIndicator, Text, View } from 'react-native';
import {
  CloudSun,
  Thermometer,
  Wind,
  CloudRain,
} from 'lucide-react-native';

import { assessRidingWeather } from '../api/weather';
import type { RidingWeather } from '../types';

type Props = {
  weather: RidingWeather | null;
  loading: boolean;
  error: string | null;
};

export function RidingWeatherPanel({
  weather,
```

```
    loading,
    error,
  }: Props) {
    if (loading) {
      return (
        <View className="items-center rounded-2xl bg-surface p-5">
          <ActivityIndicator color="#36B9F4" />
          <Text className="mt-2 text-xs text-gray-400">
            Checking riding conditions...
          </Text>
        </View>
      );
    }

    if (error || !weather) {
      return (
        <View className="rounded-2xl bg-surface p-4">
          <Text className="text-xs text-gray-400">
            {error || 'Weather unavailable.'}
          </Text>
        </View>
      );
    }

    const assessment = assessRidingWeather(weather);

    const statusColor = {
      good: '#36B9F4',
      fair: '#F5C76C',
      poor: '#EF7777',
    }[assessment.status];

    return (
      <View className="gap-4 rounded-2xl bg-surface p-4">
        <View className="flex-row items-center justify-between">
          <View>
            <Text className="text-sm font-semibold text-white">
              Riding Conditions
            </Text>
            <Text className="mt-1 text-xs text-gray-400">
              Toronto · Current weather
            </Text>
```

```
  </View>

  <CloudSun size={26} color="#36B9F4" />
</View>

<View className="flex-row items-center gap-4">
  <Text className="text-3xl font-bold text-white">
    {Math.round(weather.temperature)}°C
  </Text>

  <View className="flex-1">
    <Text
      className="text-xs font-semibold"
      style={{ color: statusColor }}
    >
      {assessment.label}
    </Text>

    <Text className="mt-1 text-xs text-gray-400">
      {assessment.description}
    </Text>
  </View>
</View>

<View className="h-px bg-white/10" />

<View className="flex-row justify-between">
  <View className="flex-1 items-center gap-2">
    <Thermometer size={18} color="#A8B0C0" />
    <Text className="text-[10px] text-gray-400">
      Feels like
    </Text>
    <Text className="text-xs text-white">
      {Math.round(weather.feelsLike)}°C
    </Text>
  </View>

  <View className="flex-1 items-center gap-2">
    <Wind size={18} color="#A8B0C0" />
    <Text className="text-[10px] text-gray-400">
      Wind
    </Text>
```

```
            <Text className="text-xs text-white">
              {Math.round(weather.windSpeed)} km/h
            </Text>
          </View>

          <View className="flex-1 items-center gap-2">
            <CloudRain size={18} color="#A8B0C0" />
            <Text className="text-[10px] text-gray-400">
              Rain
            </Text>
            <Text className="text-xs text-white">
              {weather.precipitation.toFixed(1)} mm
            </Text>
          </View>
        </View>

        <Text className="text-center text-[10px] text-gray-500">
          Weather data: Open-Meteo
        </Text>
      </View>
    );
}
```

This component should fit naturally into your existing navy/blue UI.

7. Integrate it into `product/[id].tsx`

Now we connect the API functionality to your existing product detail screen.

Add the imports:

```
import { useRidingWeather } from '@/features/weather/hooks/useRidingWeather';
import { RidingWeatherPanel } from
'@/features/weather/components/RidingWeatherPanel';
```

Inside `ProductDetailScreen`, alongside your other hooks:

```
const { weather, loading, error } = useRidingWeather();
```

Then update the Description branch of your existing `SegmentedToggle` content:

```
{selectedTab === 'description' ? (
  <View className="gap-6">
    <View>
      <Text className="text-sm font-bold text-white">
        {product.name}
      </Text>

      <Text className="mt-3 text-xs leading-5 text-gray-400">
        {product.description}
      </Text>
    </View>

    {product.category !== 'helmet' && (
      <RidingWeatherPanel
        weather={weather}
        loading={loading}
        error={error}
      />
    )}
  </View>
) : (
  <SpecsPanel specs={product.specs} />
)}
```

Everything else stays as you have it, including `BikeHero`, `AccentButton`, the cart store, and navigation.

8. Test the integration

Once the code is in place, I would verify these scenarios:

- Open a bicycle detail page and confirm the loading indicator appears before the weather data.
- Verify the temperature, wind speed, and precipitation match the API response.
- Switch between Description and Specification and confirm both panels work independently.
- Disconnect the simulator from the internet temporarily and verify the error state displays without crashing.
- Open a helmet product and confirm the cycling weather panel is hidden.

- Add a bicycle to the cart and verify the existing functionality still works.

9. How I'd explain this in your submission

The main architectural advantage is that we've separated four responsibilities:

Layer	Responsibility
API function	Communicates with Open-Meteo
Custom hook	Handles loading, errors, and API state
UI component	Displays the weather and riding assessment
Product screen	Coordinates the product and weather features

This is a clean example of integrating a third-party API without tightly coupling your UI to its response format.

One improvement I'd consider if you have time is allowing the user to select their city using Open-Meteo's geocoding API, which supports searching for locations by name. Open Meteo That would make the weather feature genuinely useful beyond the hardcoded demonstration.

My recommendation: Implement this version first and verify the API works on your simulator. Then finish polishing the Figma styling rather than expanding the weather feature further. You'll have demonstrated external API integration, modular React Native development, shared state with Zustand, navigation, and UI animation in one cohesive take-home project.

You said:

```
import { LinearGradient } from 'expo-linear-gradient';
import { useFocusEffect } from 'expo-router';
import { PropsWithChildren, useCallback, useRef } from 'react';
import { useWindowDimensions } from 'react-native';
import Animated, { cancelAnimation, Easing, ReduceMotion,
useAnimatedStyle, useSharedValue, withTiming } from 'react-
native-reanimated';
import {
  SafeAreaView,
  type Edge,
} from 'react-native-safe-area-context';
```

```
import { colors, gradients } from '@/theme/tokens';

type Props = PropsWithChildren<{
  className?: string;
  edges?: Edge[];
  gradient?: boolean;
  animate?: boolean;
}>;

export function Screen({
  children,
  className = '',
  edges = ['top', 'bottom'],
  gradient = false,
  animate = false,
}: Props) {
  const { width } = useWindowDimensions();
  const focusedBefore = useRef(false);
  const translateX = useSharedValue(animate ? width : 0);

  useFocusEffect(useCallback(() => {
    if (!animate) return;
    translateX.value = focusedBefore.current ? -width : width;
    focusedBefore.current = true;
    translateX.value = withTiming(0, {
      duration: 280,
      easing: Easing.out(Easing.cubic),
      reduceMotion: ReduceMotion.System,
    });
    return () => cancelAnimation(translateX);
  }, [animate, width, translateX]));

  const transitionStyle = useAnimatedStyle(() => ({
    transform: [{ translateX: translateX.value }],
  }));

  return (
    <Animated.View className="flex-1" style={[{ backgroundColor:
colors.bg }, transitionStyle]}>
      {gradient && (
        <LinearGradient
          {...gradients.listingBackground}
```

```
        style={{
          position: 'absolute',
          top: 0,
          bottom: 0,
          left: 0,
          right: 0,
        }}
      />
    )}

    <SafeAreaView
      edges={edges}
      className={`flex-1 ${className}`}
    >
      {children}
    </SafeAreaView>
  </Animated.View>
 );
}
```

I added some Reanimated animations here as per the instructions. Did I implement these correctly?

ChatGPT said:

Yes, **your implementation is technically sound in several important ways**, but I would change how you're handling the navigation animations before considering it complete.

You've implemented the Reanimated APIs correctly, but there's one architectural issue: **you're animating the entire `Screen` while Expo Router's native `Stack` may already be animating that same screen.**

That can produce overlapping transitions and unexpected behavior.

1. Code review

Area	Assessment	Explanation
`useSharedValue`	Correct	Initializes the horizontal position
`useAnimatedStyle`	Correct	Applies an animated translation

`withTiming`	Correct	Uses a 280 ms easing animation
`useFocusEffect`	Mostly correct	Runs the animation whenever the screen gains focus
`cancelAnimation`	Correct	Cancels an active animation during cleanup
`ReduceMotion.System`	Good	Respects system accessibility preferences
`useWindowDimensions`	Good	Accounts for different screen widths
`focusedBefore`	Needs reconsideration	Doesn't reliably distinguish forward navigation from back navigation
Native Stack interaction	Needs reconsideration	May animate the screen twice

The main issue: `focusedBefore`

You currently have:

```
translateX.value = focusedBefore.current ? -width : width;
focusedBefore.current = true;
```

Your intended behavior appears to be:

- First visit: slide in from the right.
- Returning to the screen: slide in from the left.

However, `focusedBefore.current` only tells you whether this particular component instance has previously received focus.

It doesn't tell you which direction the user navigated.

For example, if the user visits Product Details, navigates to Cart, then returns to Product Details, the animation comes from the left. But navigating from Product Details to another screen and then returning will always use that same direction, regardless of the actual navigation structure.

That might work for one specific flow, but it isn't reliable general-purpose navigation behavior.

2. My recommendation: Let Expo Router handle screen transitions

You already have a native Stack navigator.

I'd use its built-in animations:

```
<Stack
  screenOptions={{
    headerStyle: {
      backgroundColor: '#242C3C',
    },
    headerTintColor: '#FFFFFF',
    contentStyle: {
      backgroundColor: '#242C3C',
    },
    animation: 'slide_from_right',
    gestureEnabled: true,
    fullScreenGestureEnabled: true,
  }}
>
```

With this implementation:

- Pushing Product Details slides it in from the right.
- Navigating backward reverses the stack transition.
- Swipe-to-go-back gestures remain supported.
- You avoid animating the screen manually.

This is the better solution for the navigation requirement in your Figma.

In particular, native navigation animations are integrated with the navigation lifecycle, whereas `useFocusEffect` runs when the screen receives focus.

3. Simplify `Screen.tsx`

If we're letting Expo Router manage navigation transitions, I'd simplify your component to this:

```
import { LinearGradient } from 'expo-linear-gradient';
import { PropsWithChildren } from 'react';
import { View } from 'react-native';
import {
  SafeAreaView,
  type Edge,
} from 'react-native-safe-area-context';
```

```
import { colors, gradients } from '@/theme/tokens';

type Props = PropsWithChildren<{
  className?: string;
  edges?: Edge[];
  gradient?: boolean;
}>;

export function Screen({
  children,
  className = '',
  edges = ['top', 'bottom'],
  gradient = false,
}: Props) {
  return (
    <View
      className="flex-1"
      style={{ backgroundColor: colors.bg }}
    >
      {gradient && (
        <LinearGradient
          {...gradients.listingBackground}
          style={{
            position: 'absolute',
            top: 0,
            bottom: 0,
            left: 0,
            right: 0,
          }}
        />
      )}

      <SafeAreaView
        edges={edges}
        className={`flex-1 ${className}`}
      >
        {children}
      </SafeAreaView>
    </View>
  );
}
```

This also removes several imports and hooks that you no longer need.

The `Screen` component now has three responsibilities:

1. Apply the screen background.
2. Optionally render the gradient.
3. Handle safe-area insets.

That's a cleaner abstraction.

4. But what if you want custom screen animations?

There is still a valid use case for Reanimated inside individual screens.

For example, when Product Details opens, you could animate the actual content:

- Bicycle image slides upward slightly.
- Product title fades in.
- Description panel fades in.
- Add to Cart button appears last.

These are **content animations**, rather than navigation animations.

Here's a simple example you could use around your `BikeHero`:

```
import Animated, {
  FadeInUp,
} from 'react-native-reanimated';
```

Then:

```
<Animated.View entering={FadeInUp.duration(350)}>
  <BikeHero product={product} />
</Animated.View>
```

This works well alongside the native Stack because you're animating the bicycle content rather than moving the entire screen.

You can similarly animate other content:

```
<Animated.View entering={FadeInUp.delay(100).duration(350)}>
  <SegmentedToggle
    value={selectedTab}
    onChange={setSelectedTab}
  />
</Animated.View>
```

These animations are optional, but they'd complement the Figma nicely.

A caveat: entering animations can play again when content remounts, so I wouldn't apply them indiscriminately to every element.

5. One issue with `ReduceMotion.System`

You correctly included:

```
reduceMotion: ReduceMotion.System
```

That's good accessibility practice.

However, when the system requests reduced motion, Reanimated can suppress the animation. Since you initially position the screen at `width`, the final behavior depends on how Reanimated applies the reduced-motion setting to the animation.

If you ever return to a custom full-screen animation, test it with the device's Reduce Motion accessibility setting enabled.

This is another reason I'd prefer native-stack transitions for navigation.

Final verdict

I'd give your current implementation a pass for Reanimated knowledge, but I wouldn't ship the custom `Screen` navigation animation.

Your use of shared values, timing, easing, focus effects, and cleanup demonstrates that you understand the basics.

But for this project, I would use:

Animation	Best implementation
Screen navigation	Expo Router native Stack
Button press	Your `PressableScale`
Description / Specification	Your `SegmentedToggle`
Checkout drag	Your `SlideToConfirm`
Bicycle image entrance	Optional Reanimated `FadeInUp`
Product cards	Optional staggered Reanimated entrance

That gives you the same visual polish while keeping navigation behavior predictable.

My recommendation is to remove `animate` from `Screen.tsx`, keep your gradient implementation exactly as it is, and let Expo Router handle the horizontal screen transitions. Your Reanimated work is better spent on the interactive elements and subtle content transitions that make this UI distinctive.

You said:

I did implement custom screen animations, should I keep my Screen.tsx as is if this is the case?

ChatGPT said:

Yes. **If you've deliberately implemented custom screen transitions using Reanimated, you can keep your existing `Screen.tsx`.** There's no reason to discard that work if the animations are behaving as intended.

However, I'd make two adjustments before considering it finished.

1. Disable Expo Router's native animations

Since you're handling the transitions yourself, you don't want Expo Router animating the same screen simultaneously.

In your root `_layout.tsx`:

```
<Stack
  screenOptions={{
    headerStyle: { backgroundColor: '#242C3C' },
    headerTintColor: '#FFFFFF',
    contentStyle: { backgroundColor: '#242C3C' },

    animation: 'none',
  }}
>
```

Important: This applies to every screen in the Stack. If you only want custom animations for certain screens, set `animation: 'none'` on those specific routes instead.

2. Reconsider the `focusedBefore` logic

Your existing code:

```
translateX.value = focusedBefore.current ? -width : width;
focusedBefore.current = true;
```

This means:

- First time a screen receives focus → slides in from the right.
- Every subsequent focus → slides in from the left.

That is fine if it's precisely the behavior you want.

However, it doesn't actually detect navigation direction.

For example, if you navigate forward to a screen you've previously visited, it will still slide in from the left.

For your small shopping application, this might be acceptable, but I'd be aware of the limitation.

3. Keep `Screen.tsx` for custom transitions

Your current component already provides a nice API:

```
<Screen gradient animate>
  {/* Product listing */}
```

```
</Screen>


<Screen animate>
  {/* Product details */}
</Screen>


<Screen animate>
  {/* Shopping cart */}
</Screen>
```

I like that you've made both the background gradient and animation optional.

One consideration: **I wouldn't animate the product listing screen every time a user returns to it.** That can become distracting, especially when someone is navigating between products.

You can simply use:

```
<Screen gradient>
  {/* Product listing */}
</Screen>
```

And reserve `animate` for screens where the transition materially improves the experience.

My recommendation

Keep the implementation, with the native Stack animation disabled for routes using your custom transition.

Your `Screen.tsx` is modular, uses Reanimated correctly, respects reduced-motion preferences, and provides optional behavior through props.

The only architectural improvement I'd eventually consider is extracting the animation behavior into a dedicated `AnimatedScreen` component, but **that's not necessary for this take-home project**.

One final caveat: your current animation only moves the incoming screen. It won't automatically reproduce the full interactive iOS navigation transition, including the outgoing screen moving in coordination with the incoming screen or tracking a back-swipe gesture. If the Figma expects that precise behavior, native Stack transitions are a better match.

For now, if your custom animations look good on the simulator and navigation behaves correctly, I'd keep them and move on to final UI polish and testing.